



두더지 게임

Team. 뚝배기 브레이커

Index

01 프로젝트 기획

02 작업 방식

03 보드 설계

04 코드 설계

05 트러블 슈팅

06 QA

07 미니 챌린지

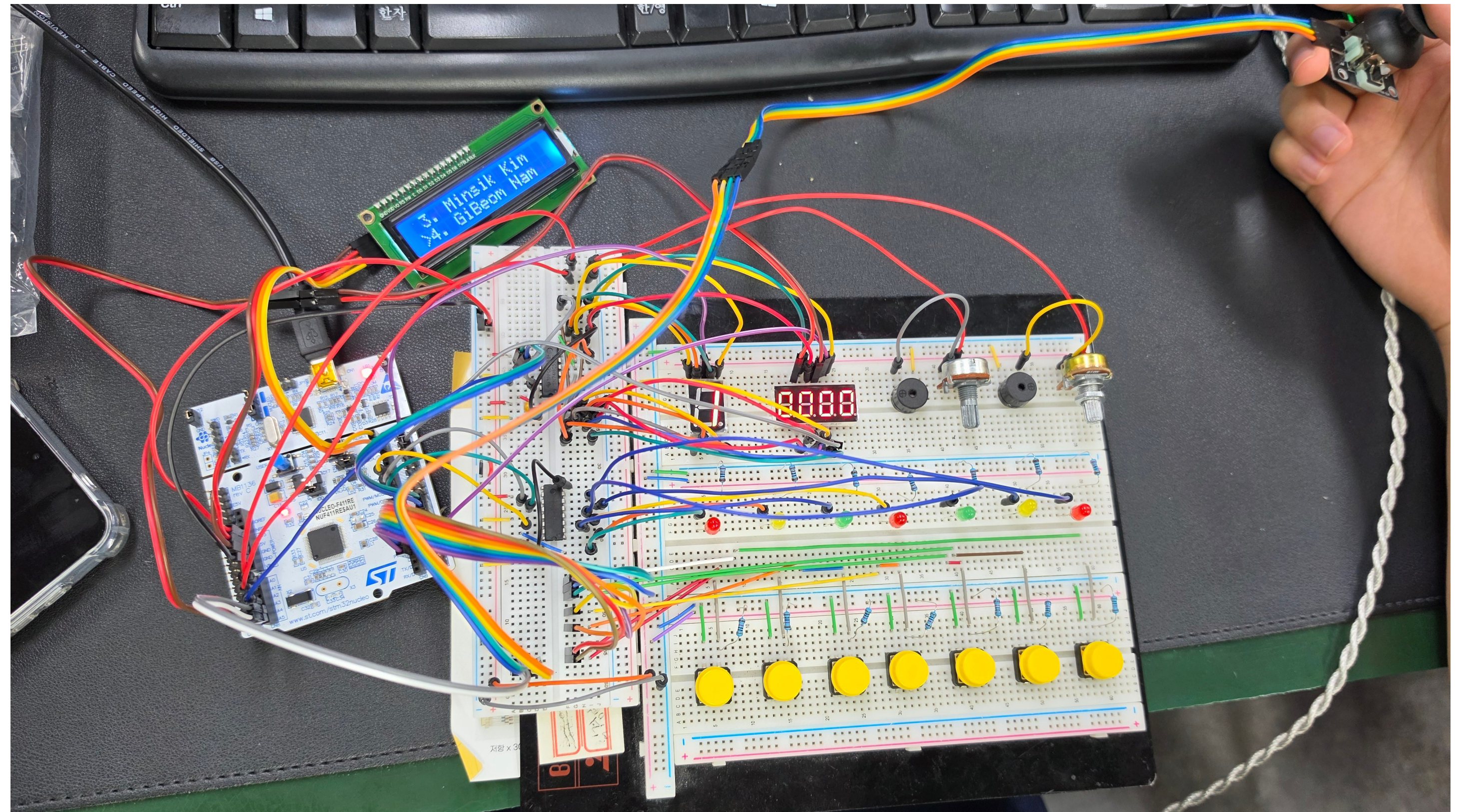


프로젝트 기획

프로젝트 소개

STM32 기반 두더지 잡기 게임

무작위로 등장하는 LED형 두더지를 잡아
점수를 획득하는 반응형 미니 게임



왜 이 주제를 선택했는가?

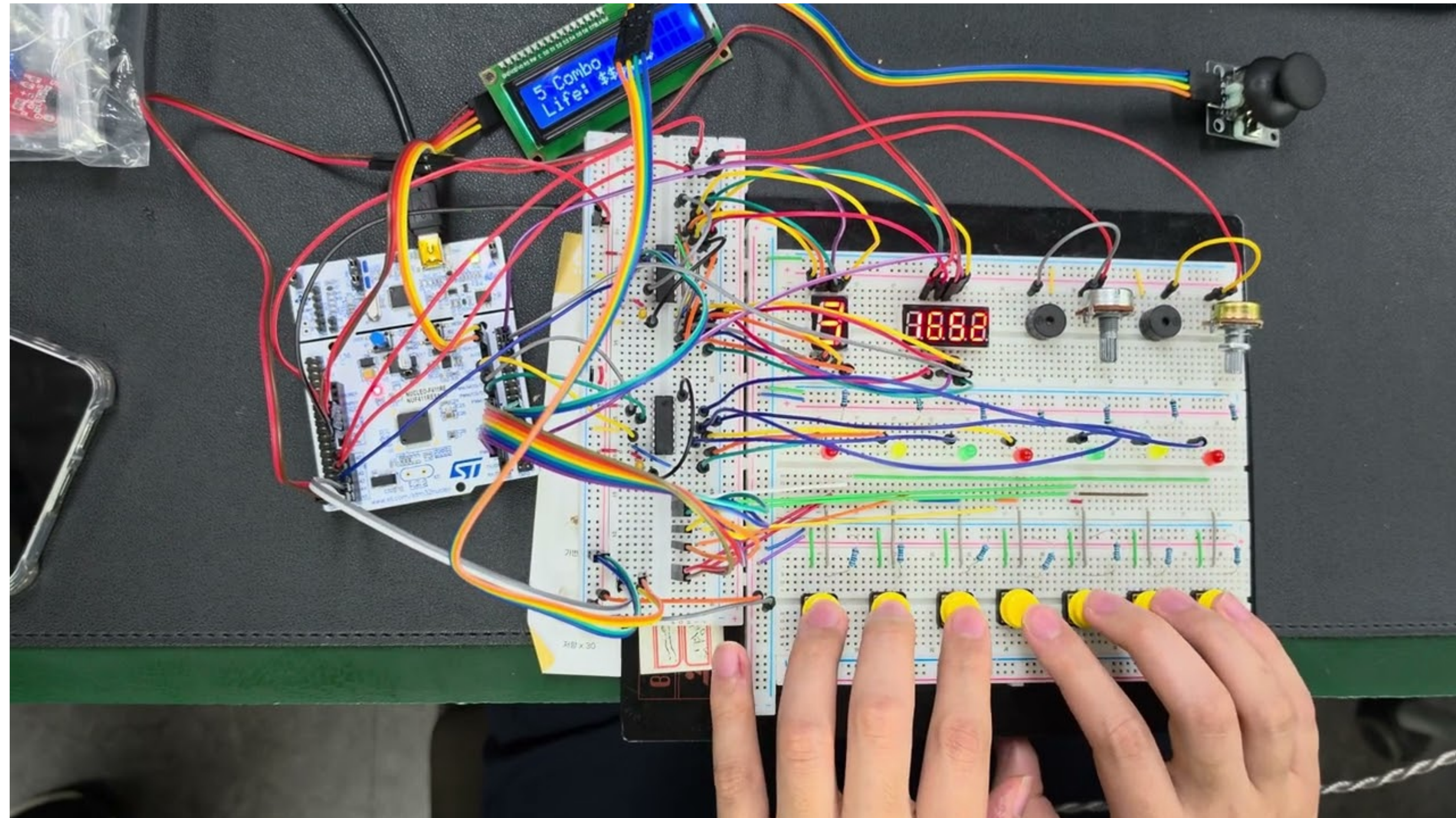
실제 두더지 게임에서 아이디어를 얻어,
수업에서 배운 STM32의 다양한 기능을 직접 응용하고
하나의 시스템으로 통합해 보고자 주제로 선정했습니다.

또한 제한된 임베디드 환경에서 게임을 구현하며
자원 관리와 최적화를 경험하는 것을 목표로 했습니다.



시연 영상

기획 설명 전에 먼저 어떻게 플레이되는지 보여드립니다.



게임 규칙

LED가 켜진 위치의 버튼을 제한시간 안에 누르면 점수와 콤보가 올라갑니다

[타격]

LED 1~7중 켜진 위치와 같은 번호의 버튼을 눌러 두더지를 잡습니다

[Miss]

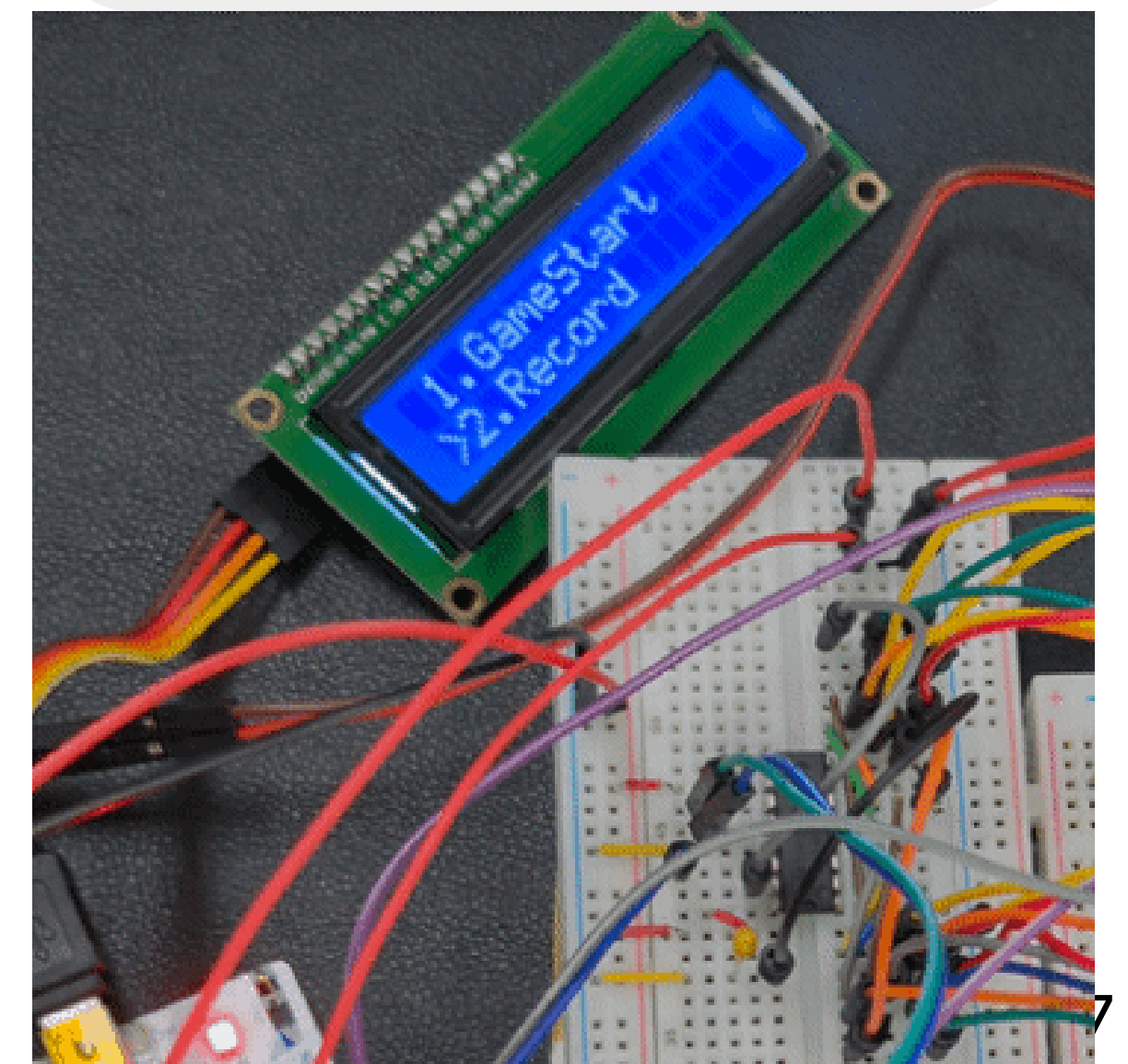
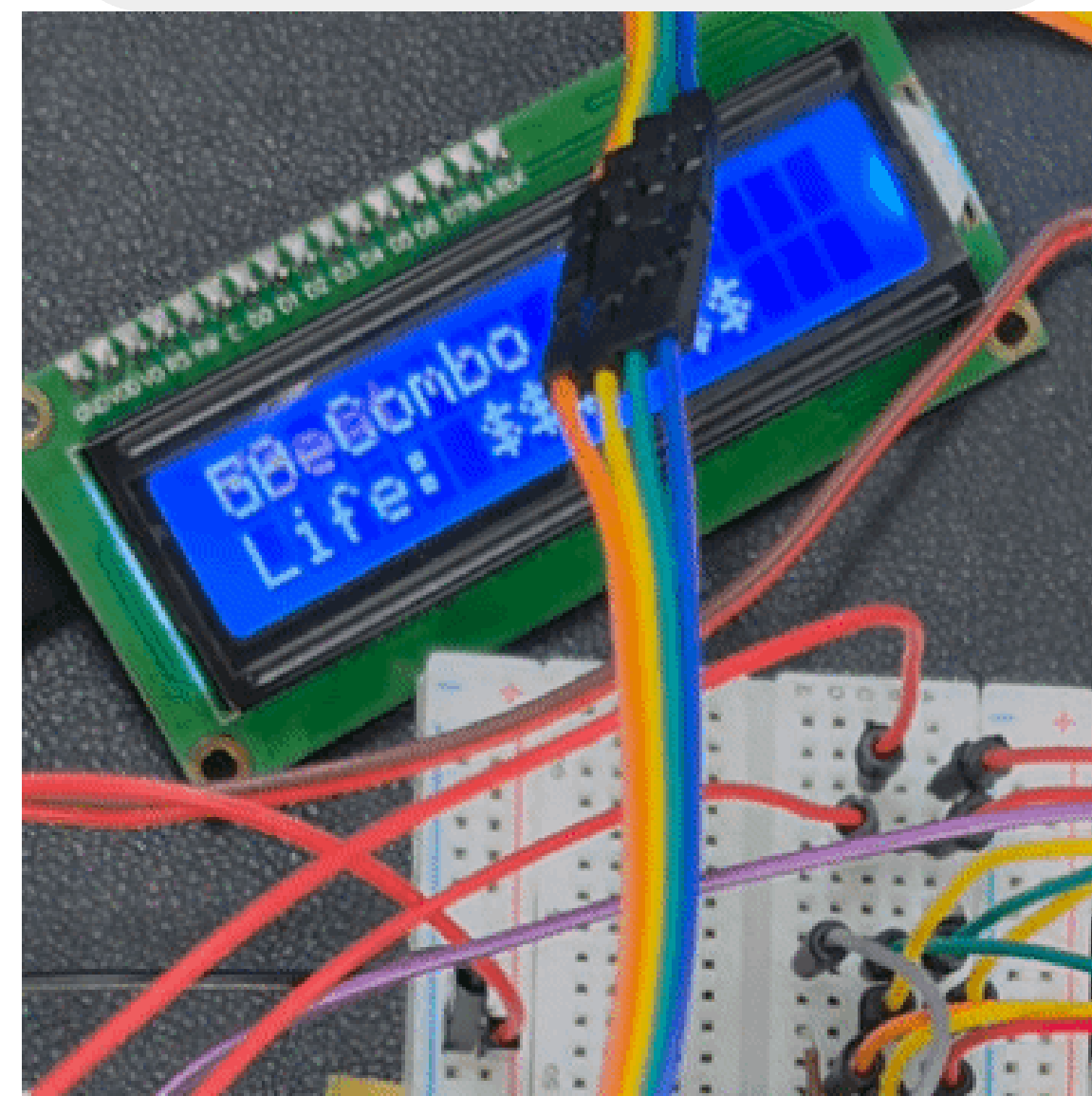
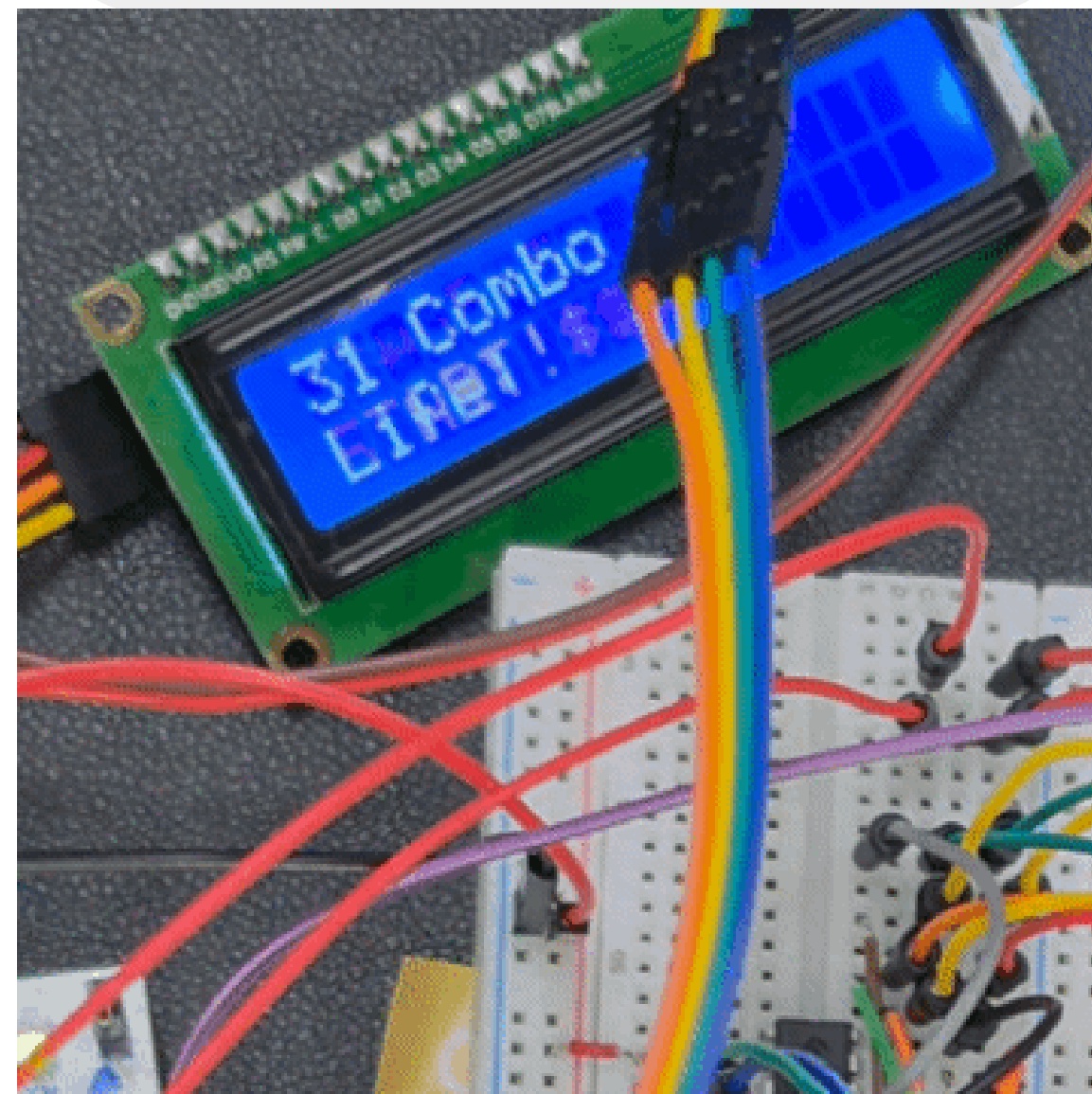
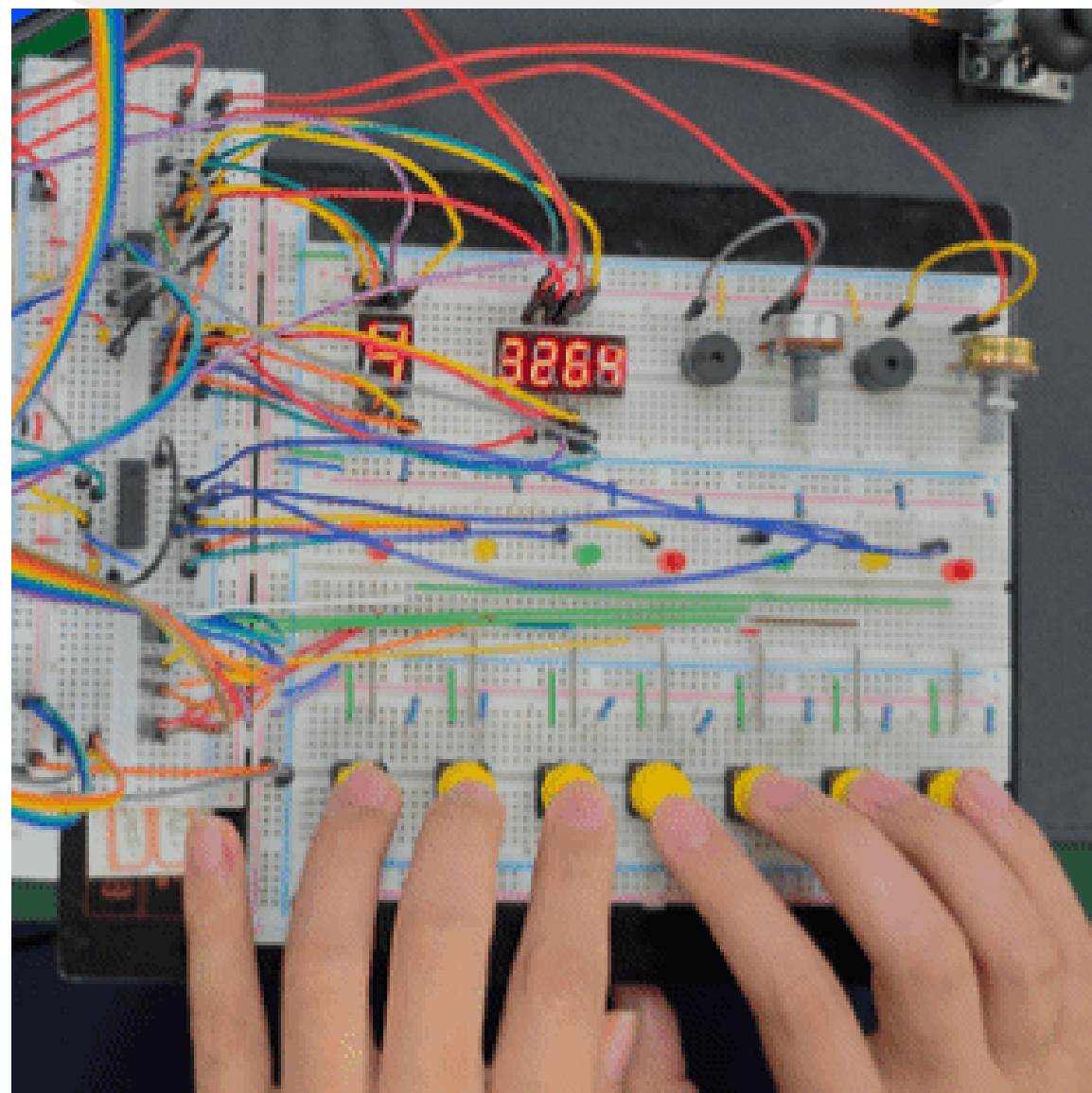
틀린 버튼을 누르거나 시간 초과 시 Life와 점수가 차감됩니다

[콤보]

Miss없이 계속 타격하는 경우 콤보 보너스를 얻습니다.

[결과]

게임이 끝나고 나면 결과가 기록되며 In100에 들 시 메모리에 기록됩니다.



스테이지 난이도

 스테이지	 시간(S)	 최대 나오는 두더지 수	 Life
1 스테이지 1 	 20	 1	 10
2 스테이지 2 	 25	 2	 8
3 스테이지 3 	 30	 3	 7
4 스테이지 4 	 60	 5	 6
5 스테이지 5 	 180	 7	 5

작업 방식

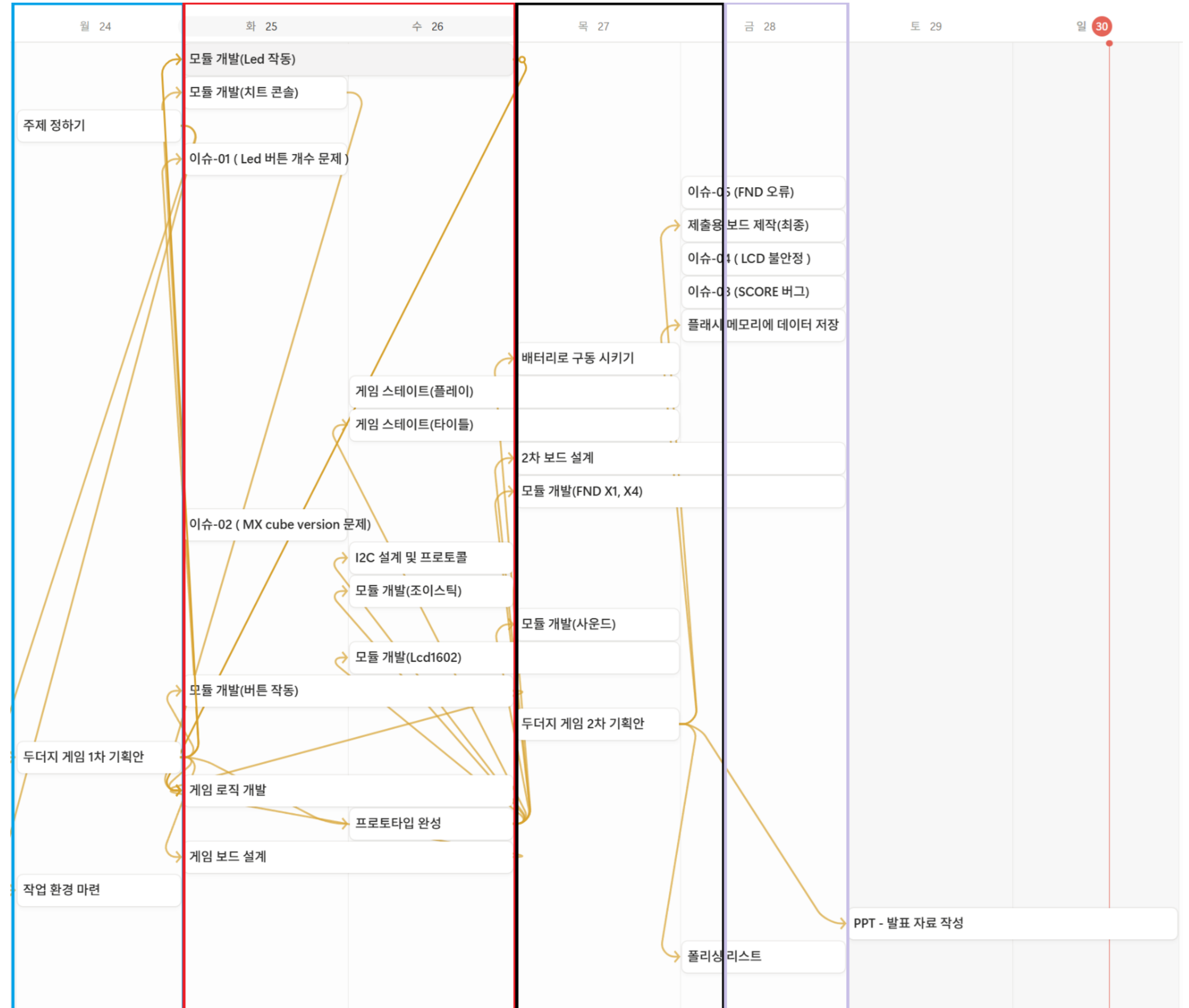
개발 일정

8월 24일 · 프로젝트 기획

8월 25~26일 · 1차 프로토타입 개발

8월 27~28일 · 2차 프로토타입 개발

8월 28일 · 기능 개선 및 최종 결과물 완성



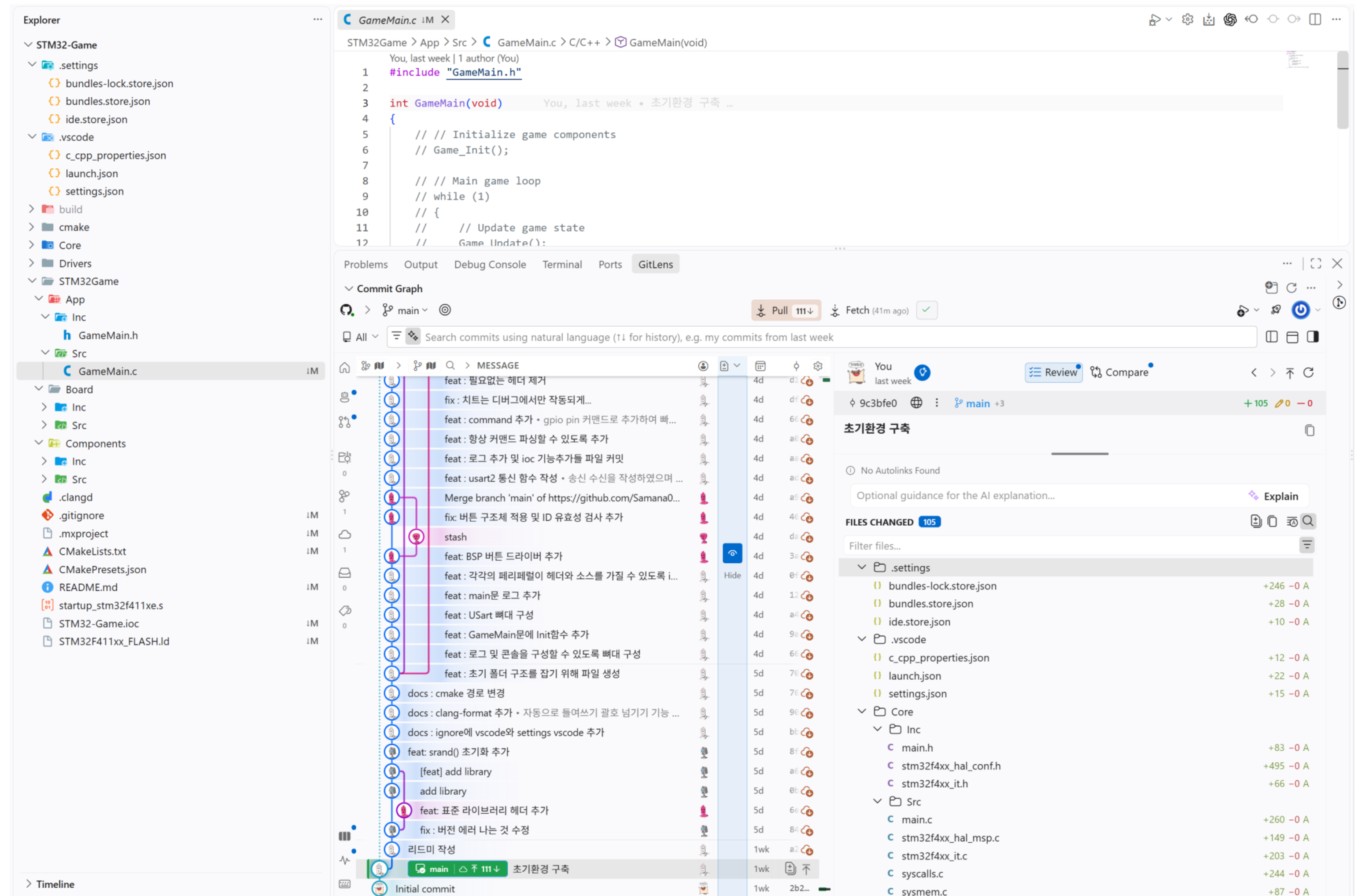
환경

우선은 개발환경 부터 마련하자

원활한 협업을 위해 팀원 간 개발 환경을 통일했습니다.

공통 빌드 환경을 구축한 뒤,
GitLens를 활용하여 Git 기반 협업 환경을 구성했습니다.

마지막으로 각 팀원의 환경에서
동일한 프로젝트의 정상 빌드를 확인한 후
본격적인 개발을 시작했습니다.



코드 스타일 통일

ClangFormat을 활용한 코드 스타일 자동화

팀원 간 일관된 코드 스타일을 유지하기 위해
공통 코딩 규칙을 정의했습니다.

ClangFormat 설정을 공유하여
코드 포맷을 자동으로 통일하도록 구성했습니다.

```
.clang-format > abc # Indent with four spaces; do not emit tab characters.
samana0104, 5 days ago | 1 author (samana0104)
1 BasedOnStyle: LLVM
2
3 ✓ # Put opening braces on the following line (Allman style).
4 BreakBeforeBraces: Allman
5
6 ✓ # Indent with four spaces; do not emit tab characters.
7 IndentWidth: 4
8 ContinuationIndentWidth: 4
9 TabWidth: 4
10 UseTab: Never
11
```

ClangFormat 설정

```
✓ static void MX_USART2_UART_Init(void) {
✓ if (HAL_UART_Init(&huart2) != HAL_OK) {
    Error_Handler();
}
/* USER CODE BEGIN USART2_Init 2 */

/* USER CODE END USART2_Init 2 */
}
```

STM32 기본 코드 스타일

```
✓ int GameMain(void)
✓ {
    GameInit();
    G_LOG(level: INFO, args: "Game started successfully.\r\n");

    while (1)
    {
        GameUpdate();
        HAL_Delay(Delay: 1);
    }

    return 0;
}
```

ClangFormat 적용 후

디버깅 환경

개발 중 GPIO 상태를 실시간으로 확인하고 직접 제어할 수 있도록 CLI 디버깅 콘솔을 구현했습니다.

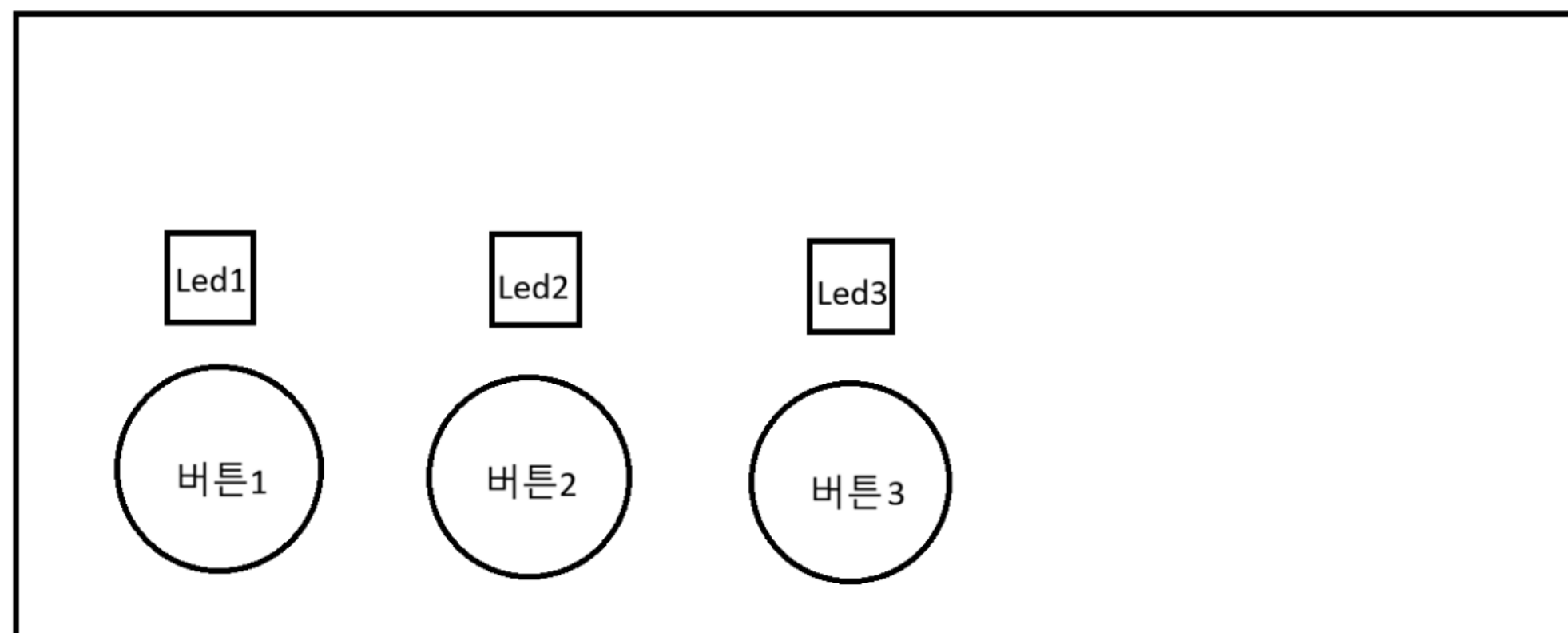
명령어를 통해 GPIO 제어 및 상태 확인, FPS 모니터링이 가능합니다.

```
[INFO] ReadyState exited.
[INFO] InGameState entered.
[INFO] InGameState exited.
[INFO] ResultState entered.
[INFO] ResultState exited.
[INFO] TitleState entered.
[INFO] TitleState exited.
[INFO] RecordState entered.
[INFO] RecordState exited.
[INFO] TitleState entered.
[INFO] Commands:
[INFO] help
[INFO] write <A|B|C|H> <0-15> <0|1>
[INFO] read <A|B|C|H> <0-15>
[INFO] led <1-7|all> <0|1>
[INFO] fps
[INFO] stop
[WARNING] write only works on pins configured as GPIO output.
[INFO] FPS: 0.000
[INFO] FPS monitoring started (every 1 second).
[INFO] FPS: 495.009
[INFO] FPS: 495.504
[INFO] FPS: 495.000
[INFO] FPS: 495.000
[INFO] FPS: 495.000
[INFO] FPS: 495.000
[INFO] FPS: 495.000
[INFO] FPS: 495.000
```

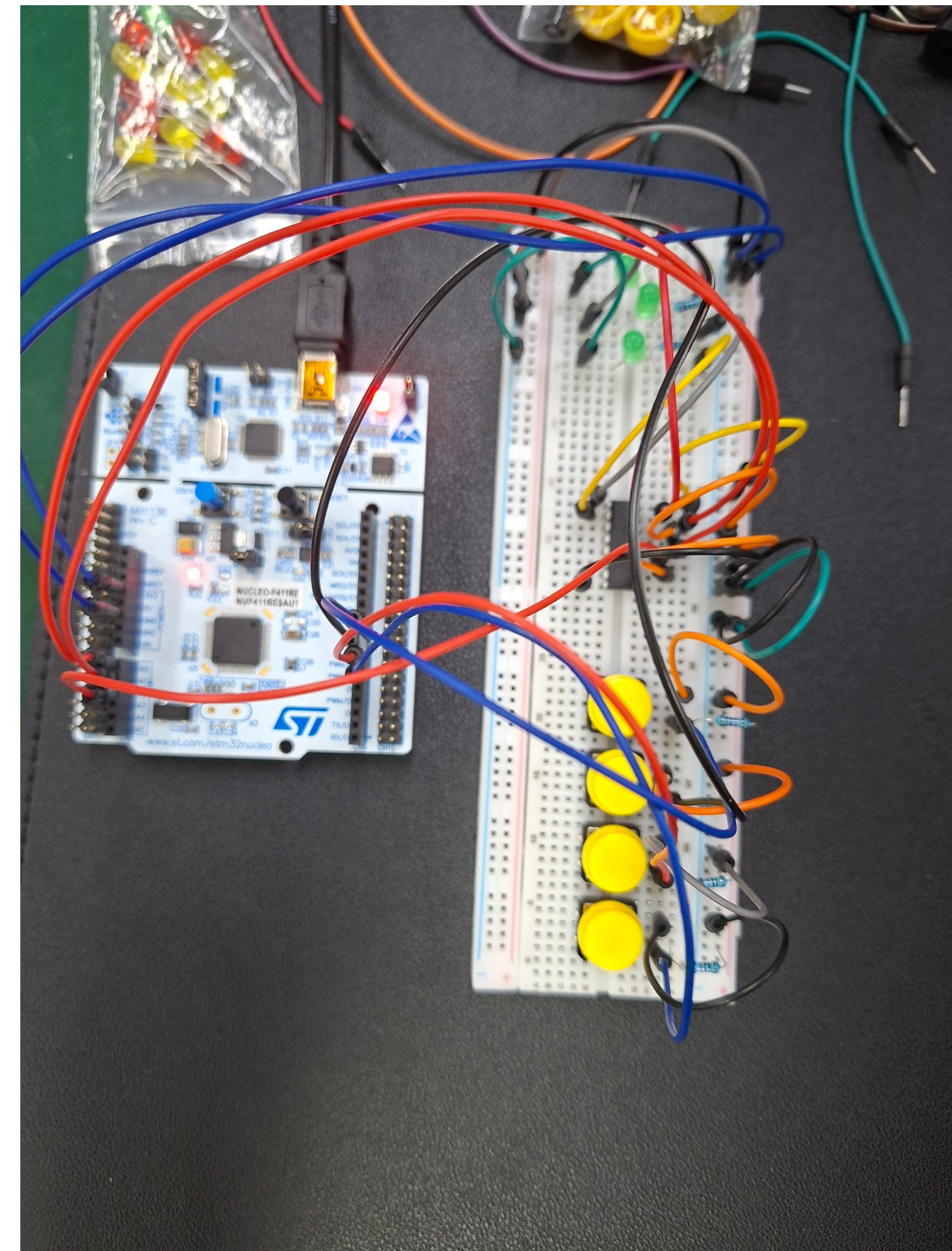

결과물부터 만들어보자

처음부터 복잡하게 구성하기보다,
눈에 보이는 결과물을 빠르게 만드는 것을 우선했습니다.

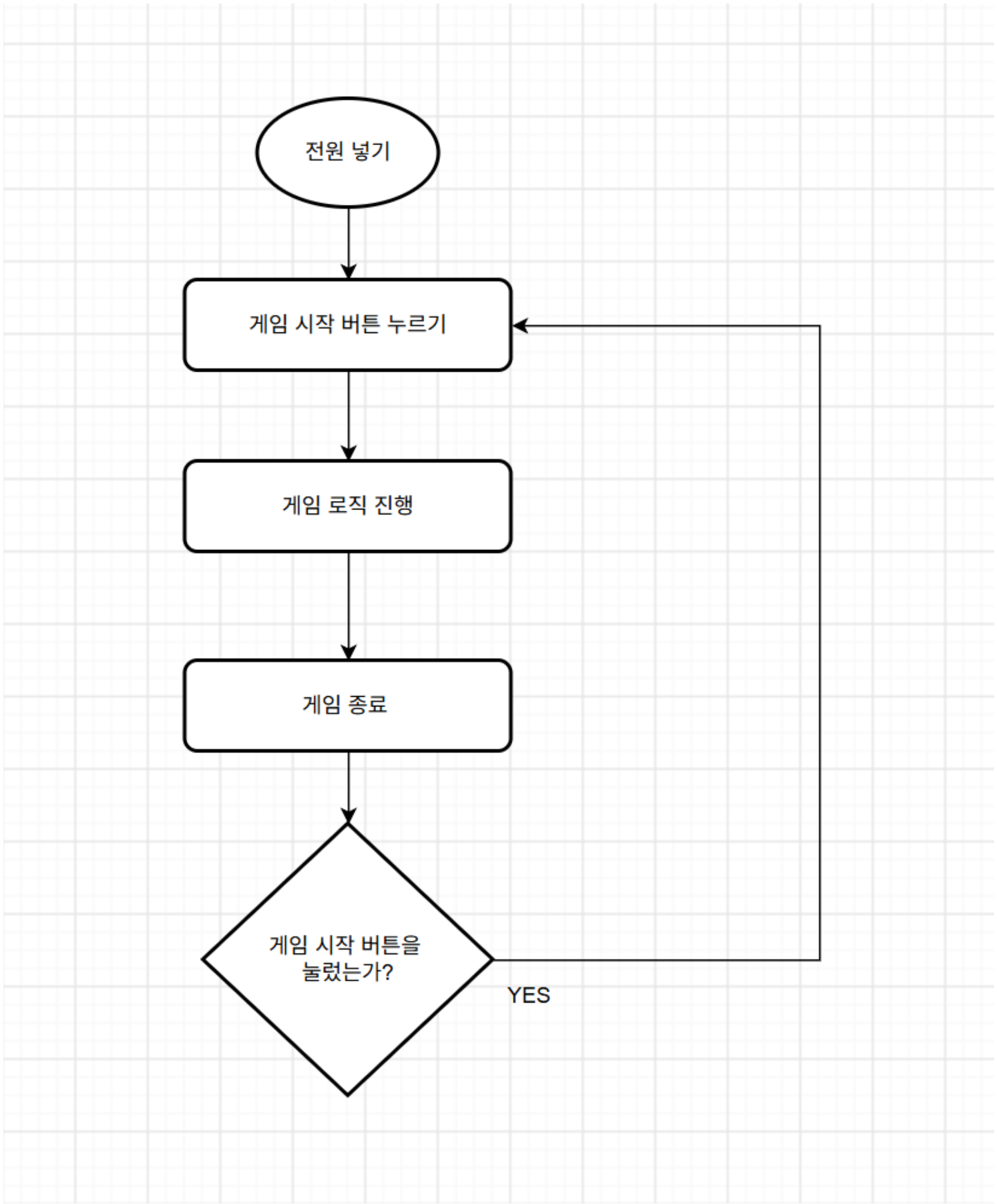
간단한 구조로 시작해 동작을 확인한 뒤,
기능을 하나씩 확장하는 방식으로 개발을 진행했습니다.



초기 프로토타입 구상도

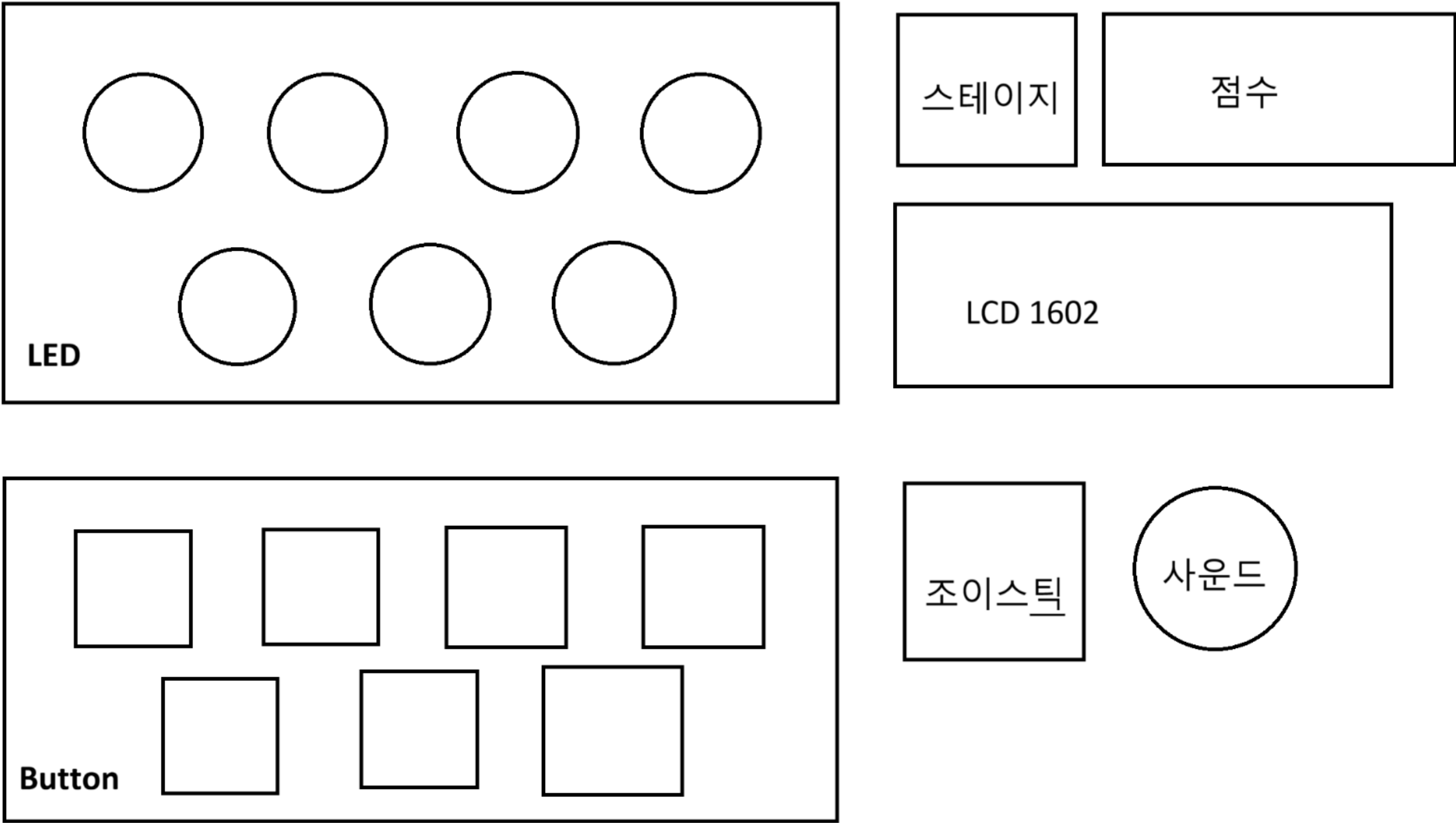


초기 프로토타입



초기 동작 플로우차트

프로토타입이 나온 이후에는
기능을 하나씩 추가하며 점진적으로 확장해 나갔습니다.



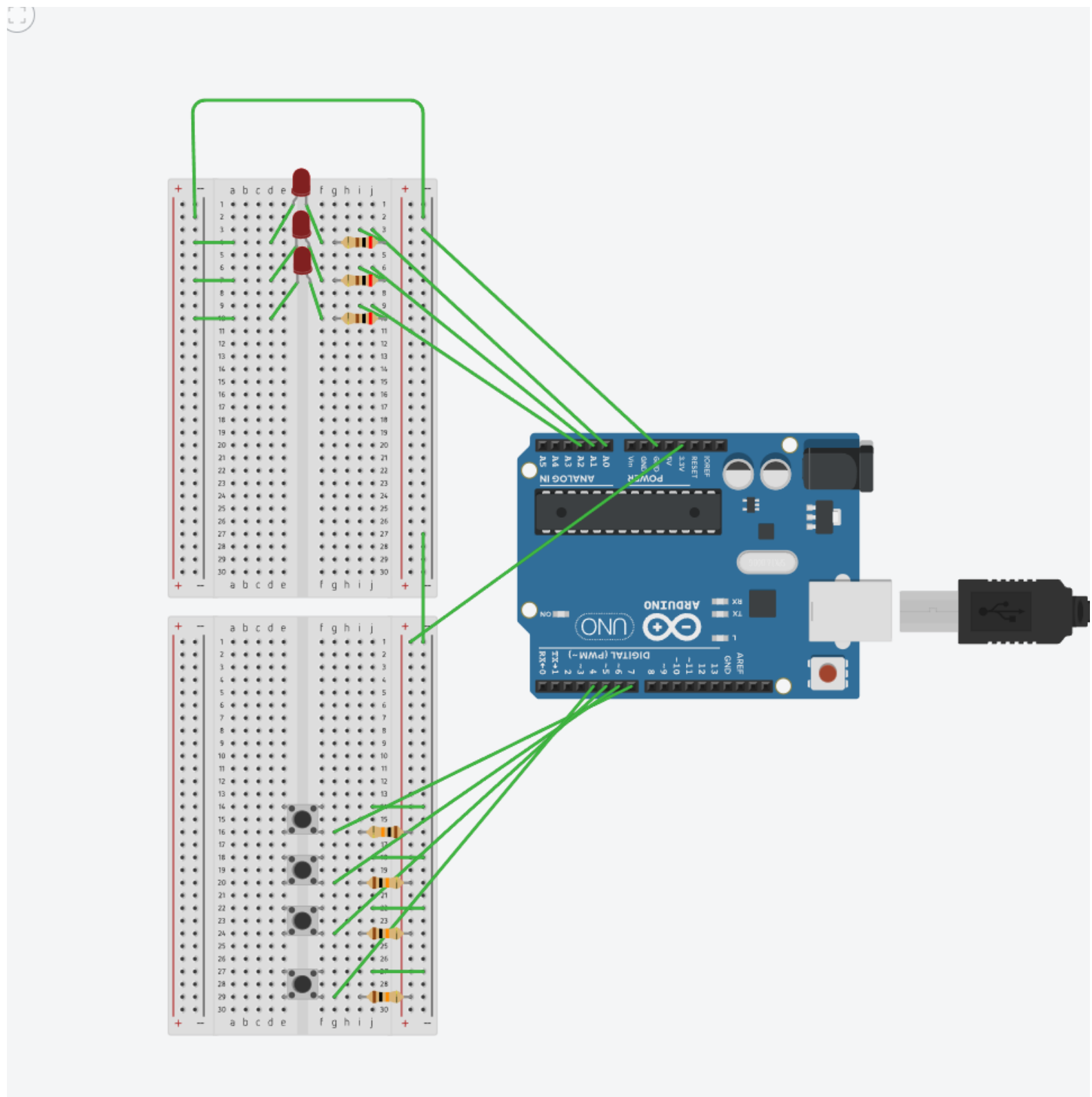
2차 프로토타입 구상도



보드 설계

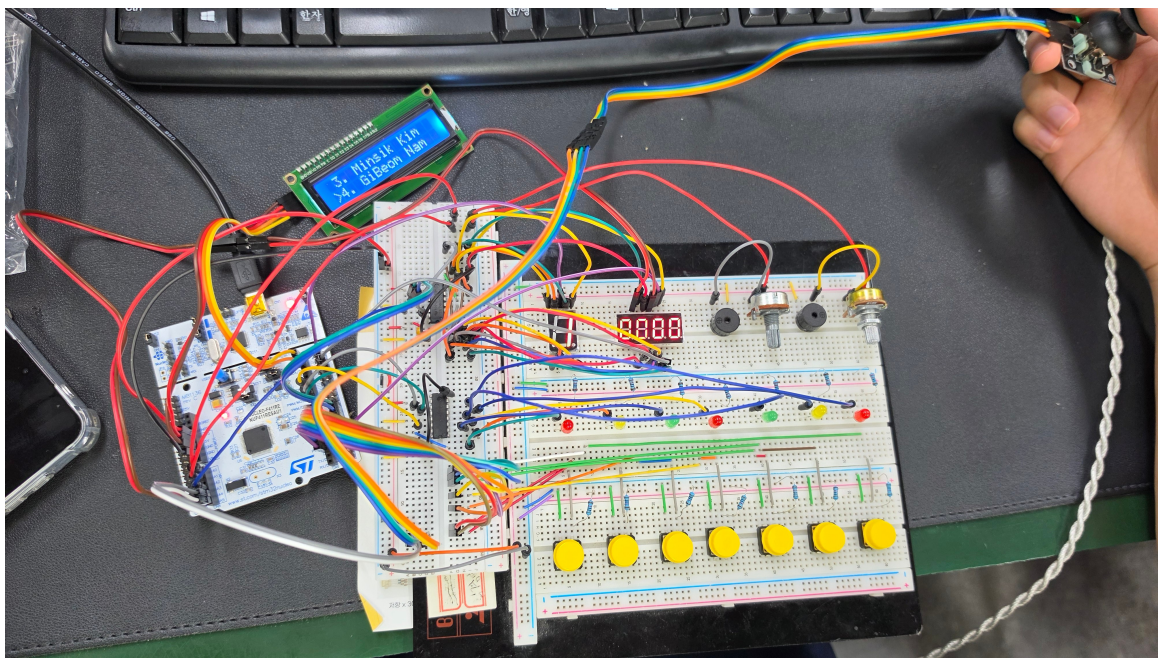
보드 제작 과정

초기 배선 → 기능 추가 → 최종 보드로 단계적으로 확장



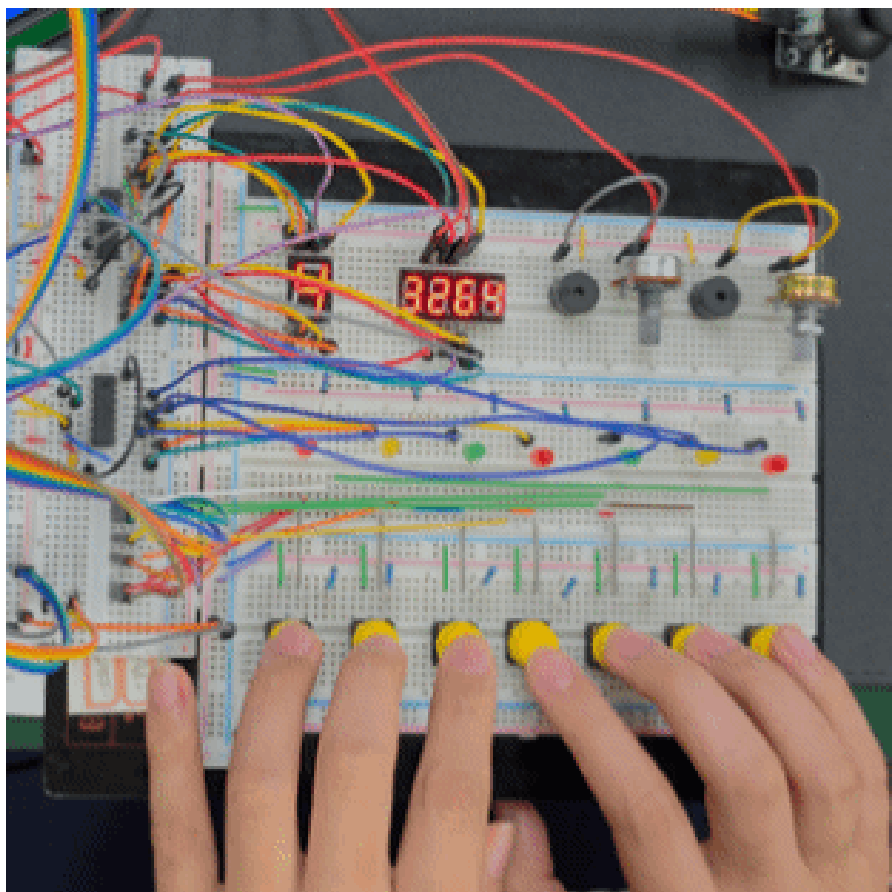
초기 배선

버튼 등 핵심 입출력을 먼저 연결하고 LED기능을 하나씩 추가하며 배선과 핀 동작을 확인했습니다



기능이 늘어날수록 배선 복잡도와 타이밍 문제가 커져 코드 계층 분리와 주기 제어가 중요해졌습니다

최종 보드



작동 영상

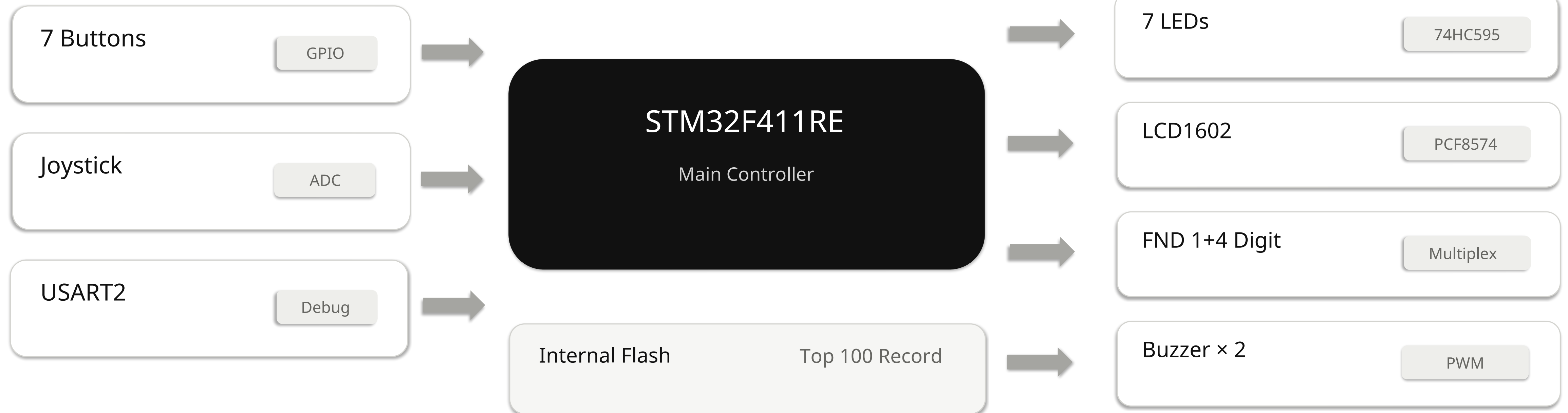
Hardware 구성

입력 · 제어 · 출력 장치를 STM32F411RE 에 통합

INPUT

CONTROL

OUTPUT



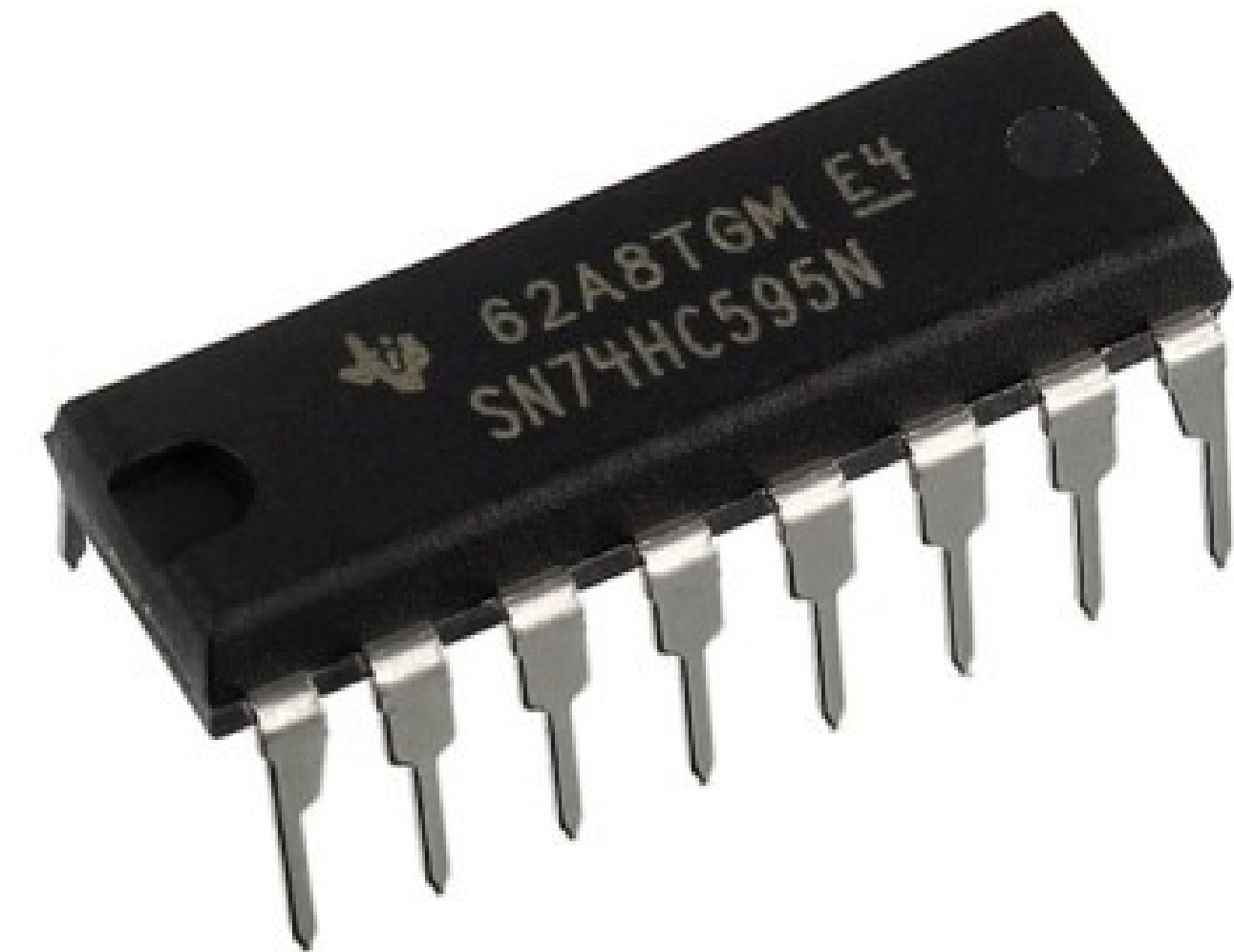
74HC595

3개의 핀으로 8개의 효과를

SIPO (Serial-In, Parallel-Out) 방식입니다.

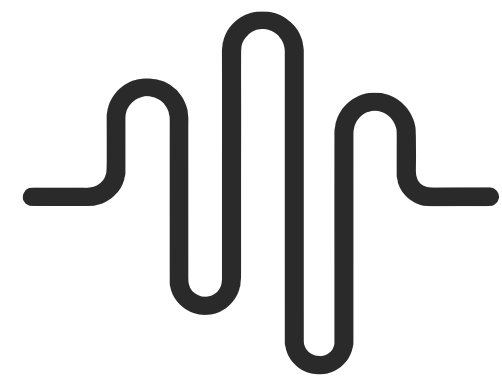
3개 핀으로 8개 핀 출력이 가능하여 핀 개수 절약이 가능합니다.

출력을 입력 데이터 삼아 여러 개로 출력 확장 가능합니다.



74HC595

3개의 핀으로 8개의 효과를



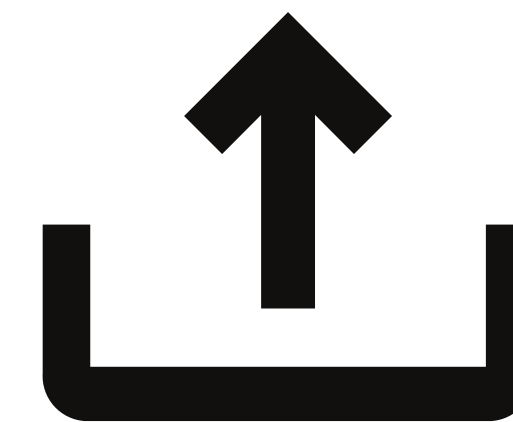
1. DATA (DS)

출력하고자 하는 0(LOW) 또는 1(HIGH)의 상태를 세팅합니다.



2. SHIFT (SHCP)

클록 펄스를 주어 세팅된 데이터를 내부 시프트 레지스터로 한 칸 밀어 넣습니다.



3. LATCH (STCP)

8칸의 방이 모두 채워지면, 래치 펄스를 가해 8개의 출력 핀(Q0~Q7)으로 동시에 쏘아냅니다.

1. 비트 세팅 → 2. 시프트(밀어넣기) × 8회 반복 → 3. 래치(동시 출력)



코드 설계

기능별 폴더 분리

게임 로직과 하드웨어 제어를 분리하여 구조화

STATE

Title · Ready · Playing · Result · Record · Credit

APP

GameMain · StageConfig · Record · SoundPlayer

DRIVER

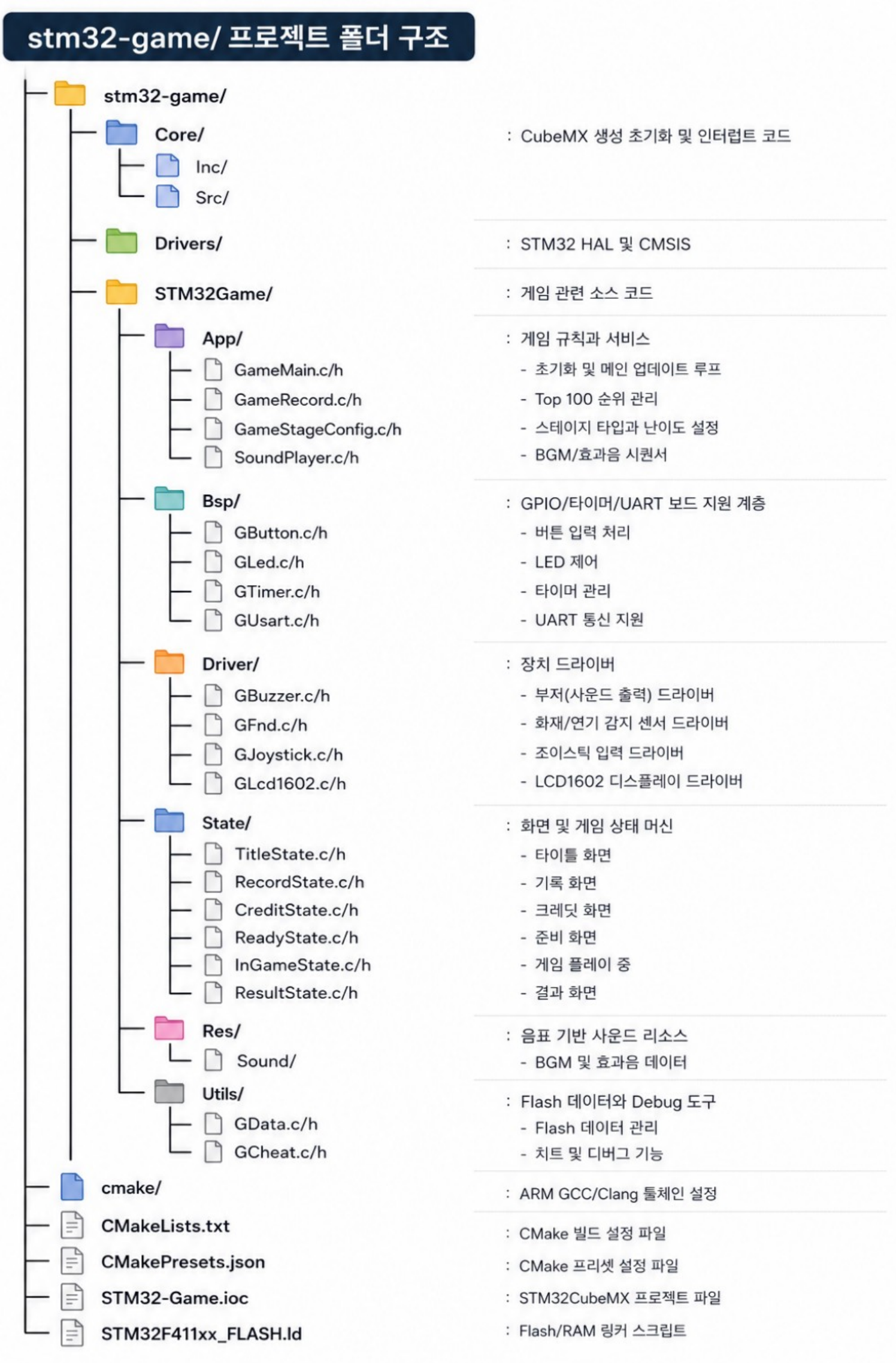
GFnd · GLcd1602 · GJoystick · GBuzzer

BSP

GButton · GLed · GTimer · GUsart

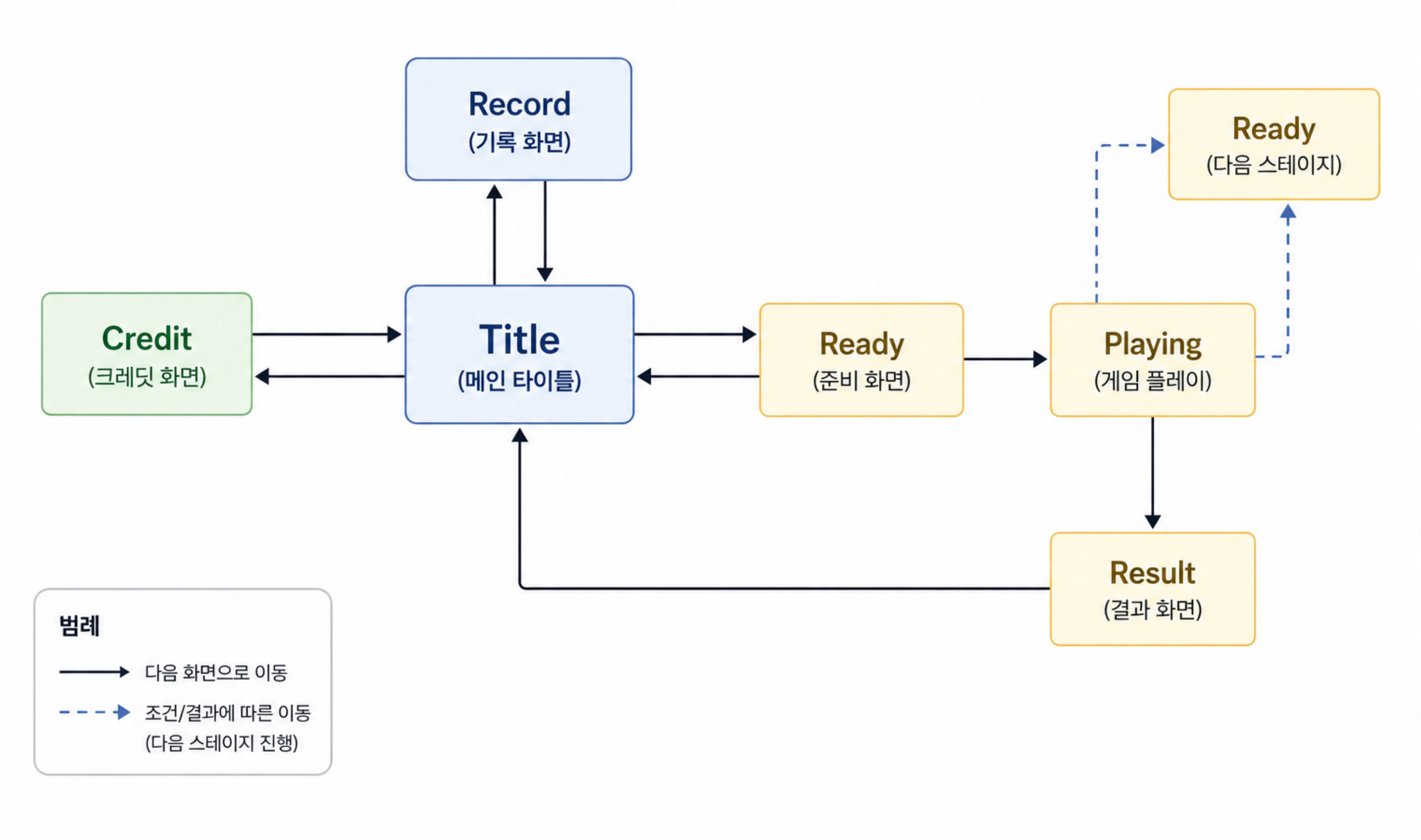
UTILS

GCheat, GData



State Machine 흐름 관리

게임 진행 상태를 분리하여 흐름을 명확하게 관리



함수 포인터 기반 Handler를 구성하여
각 State의 Enter · Update · Exit 동작을 통합 관리합니다.

```
typedef void (*GameStateFunction)(void);

...

typedef struct
{
    GameStateFunction enter;
    GameStateFunction update;
    GameStateFunction exit;
} GameStateHandler;

static const GameStateHandler stateHandlers[GAME_STATE_COUNT] =
{
    [GAME_STATE_TITLE]    = { TitleStateEnter, TitleStateUpdate, TitleStateExit },
    [GAME_STATE_RECORD]   = { RecordStateEnter, RecordStateUpdate, RecordStateExit },
    [GAME_STATE_CREDIT]   = { CreditStateEnter, CreditStateUpdate, CreditStateExit },
    [GAME_STATE_READY]    = { ReadyStateEnter, ReadyStateUpdate, ReadyStateExit },
    [GAME_STATE_PLAYING]  = { InGameStateEnter, InGameStateUpdate, InGameStateExit },
    [GAME_STATE_RESULT]   = { ResultStateEnter, ResultStateUpdate, ResultStateExit }
};
```

외부에서는
GameStateChange() 호출만으로
원하는 State로 전환할 수 있도록 설계했습니다.

이전 State Exit 호출

다음 State Enter 호출

```
--
57  bool GameStateChange(GameState nextState)
58  {
59      if (!GameStateIsValid(state: nextState))
60      {
61          return false;
62      }
63
64      if (nextState == currentState)
65      {
66          return true;
67      }
68
69      if (stateHandlers[currentState].exit != NULL)
70      {
71          stateHandlers[currentState].exit();
72      }
73
74      currentState = nextState;
75
76      if (stateHandlers[currentState].enter != NULL)
77      {
78          stateHandlers[currentState].enter();
79      }
80
81      return true;
82  }
83
```

samana0104, 4 days ago • feat : 게임 스테이트 제작

Sound

부저 주파수 제어

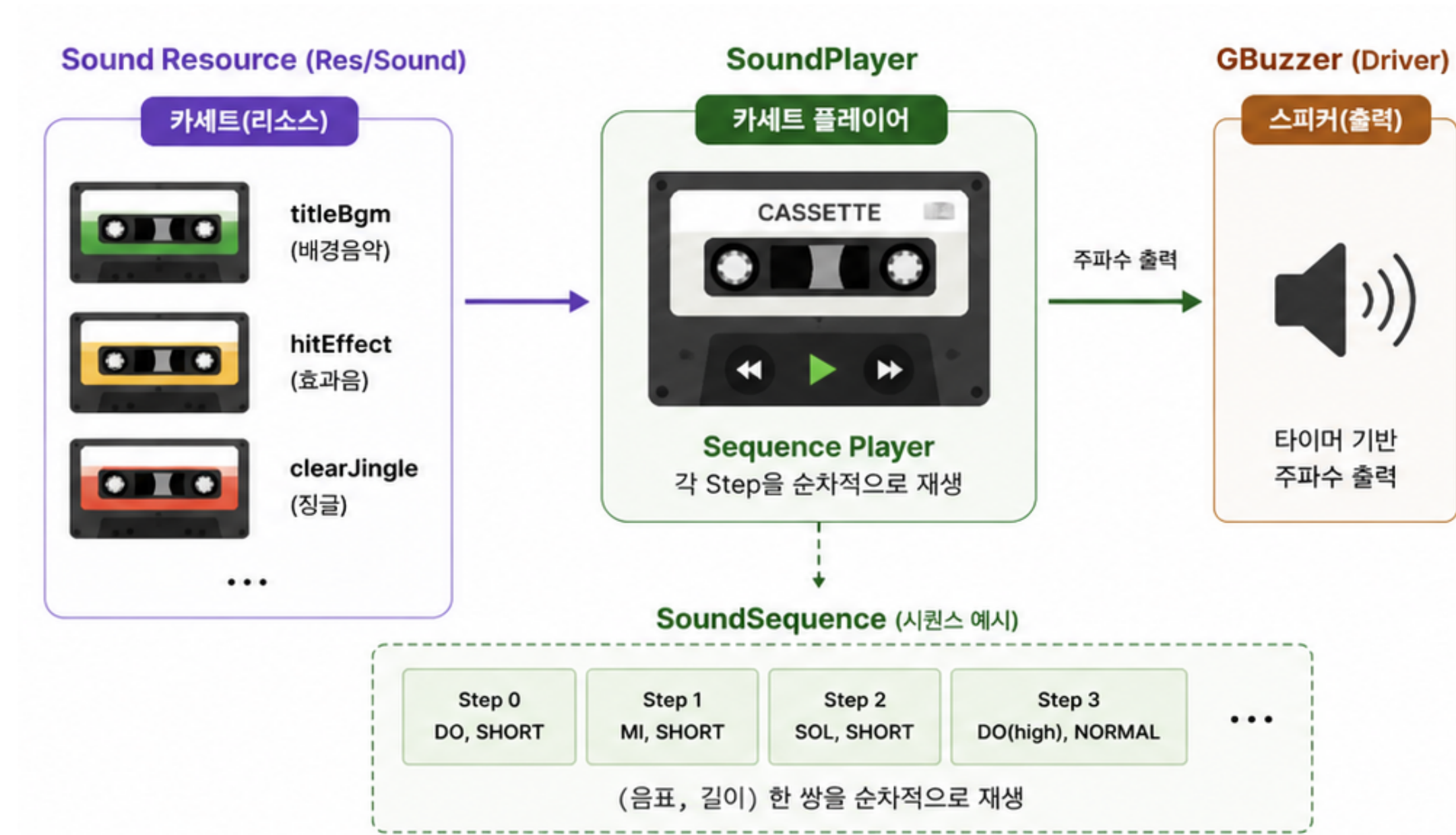
효과음과 BGM을 독립적으로 출력하기 위해
2개의 부저 모듈로 구성했습니다.

PlayFrequency()에 출력 채널과 주파수를 전달하면
Timer를 설정하여 해당 주파수의 PWM 신호를 출력합니다.

```
samana0104, 3 days ago | 1 author (samana0104)
1  #pragma once
2
3  #include "main.h"
4
samana0104, 3 days ago | 1 author (samana0104)
5  typedef enum
6  {
7      BUZZER_OUTPUT_SOUND = 0,
8      BUZZER_OUTPUT_EFFECT_SOUND,
9      BUZZER_OUTPUT_MAX_COUNT
10 } BuzzerOutput;
11
12 void PlayFrequency(BuzzerOutput output, uint32_t frequencyHz);
13 void StopBuzzer(BuzzerOutput output);
14
```

SoundPlayer

음계와 재생 시간을 Sequence로 구성하여 등록된 순서에 따라 사운드를 재생합니다.



```
samana0104, 3 days ago | 1 author (samana0104)
typedef enum
{
    SOUND_NOTE_REST = 0,
    SOUND_NOTE_LOW_RE = 147,
    SOUND_NOTE_LOW_FA = 175,
    SOUND_NOTE_LOW_SOL = 196,
    SOUND_NOTE_LOW_LA = 220,
    SOUND_NOTE_LOW_SI = 247,
    SOUND_NOTE_DO = 262,
    SOUND_NOTE_RE = 294,
    SOUND_NOTE_MI = 330,
    SOUND_NOTE_FA = 349,
    SOUND_NOTE_SOL = 392,
    SOUND_NOTE_LA = 440,
    SOUND_NOTE_SI = 494,
    SOUND_NOTE_HIGH_DO = 523,
    SOUND_NOTE_HIGH_RE = 587,
    SOUND_NOTE_HIGH_MI = 659,
    SOUND_NOTE_HIGH_FA = 698,
    SOUND_NOTE_HIGH_SOL = 784,
    SOUND_NOTE_HIGH_LA = 880,
    SOUND_NOTE_HIGH_SI = 988
} SoundNote;

samana0104, 2 days ago | 1 author (samana0104)
typedef enum
{
    SOUND_DELAY_VERY_SHORT = 75,
    SOUND_DELAY_SHORT = 100,
    SOUND_DELAY_NORMAL = 200,
    SOUND_DELAY_LONG = 400,
    SOUND_DELAY_HALF_SECOND = 500,
    SOUND_DELAY_VERY_LONG = 800,
    SOUND_DELAY_ONE_SECOND = 1000
} SoundDelay;
```

음계 및 재생 시간 정의

```
samana0104, 3 days ago | 1 author (samana0104)
#include "SoundResource.h"

static const SoundStep steps[] =
{
    [0]={SOUND_NOTE_DO, SOUND_DELAY_SHORT},
    [1]={SOUND_NOTE_MI, SOUND_DELAY_SHORT},
    [2]={SOUND_NOTE_SOL, SOUND_DELAY_SHORT},
    [3]={SOUND_NOTE_HIGH_DO, SOUND_DELAY_NORMAL},
    [4]={SOUND_NOTE_SOL, SOUND_DELAY_SHORT},
    [5]={SOUND_NOTE_MI, SOUND_DELAY_SHORT},
    [6]={SOUND_NOTE_RE, SOUND_DELAY_SHORT},
    [7]={SOUND_NOTE_FA, SOUND_DELAY_SHORT},
    [8]={SOUND_NOTE_LA, SOUND_DELAY_SHORT},
    [9]={SOUND_NOTE_HIGH_RE, SOUND_DELAY_NORMAL},
    [10]={SOUND_NOTE_LA, SOUND_DELAY_SHORT},
    [11]={SOUND_NOTE_FA, SOUND_DELAY_SHORT},
    [12]={SOUND_NOTE_MI, SOUND_DELAY_SHORT},
    [13]={SOUND_NOTE_SOL, SOUND_DELAY_SHORT},
    [14]={SOUND_NOTE_SI, SOUND_DELAY_SHORT},
    [15]={SOUND_NOTE_HIGH_MI, SOUND_DELAY_NORMAL},
    [16]={SOUND_NOTE_HIGH_RE, SOUND_DELAY_SHORT},
    [17]={SOUND_NOTE_SI, SOUND_DELAY_SHORT},
    [18]={SOUND_NOTE_SOL, SOUND_DELAY_SHORT},
    [19]={SOUND_NOTE_MI, SOUND_DELAY_SHORT},
    [20]={SOUND_NOTE_HIGH_DO, SOUND_DELAY_SHORT},
    [21]={SOUND_NOTE_SOL, SOUND_DELAY_SHORT},
    [22]={SOUND_NOTE_MI, SOUND_DELAY_SHORT},
    [23]={SOUND_NOTE_DO, SOUND_DELAY_NORMAL},
};

const SoundSequence titleBgm =
{
    steps,
    SOUND_STEP_COUNT(steps),
    true,
};
```

사운드 시퀀스 정의

사운드 리소스를 BGM과 효과음으로 구분하고,
각각 독립된 Sequence로 관리합니다.



Flash

게임 데이터의 비휘발성 저장

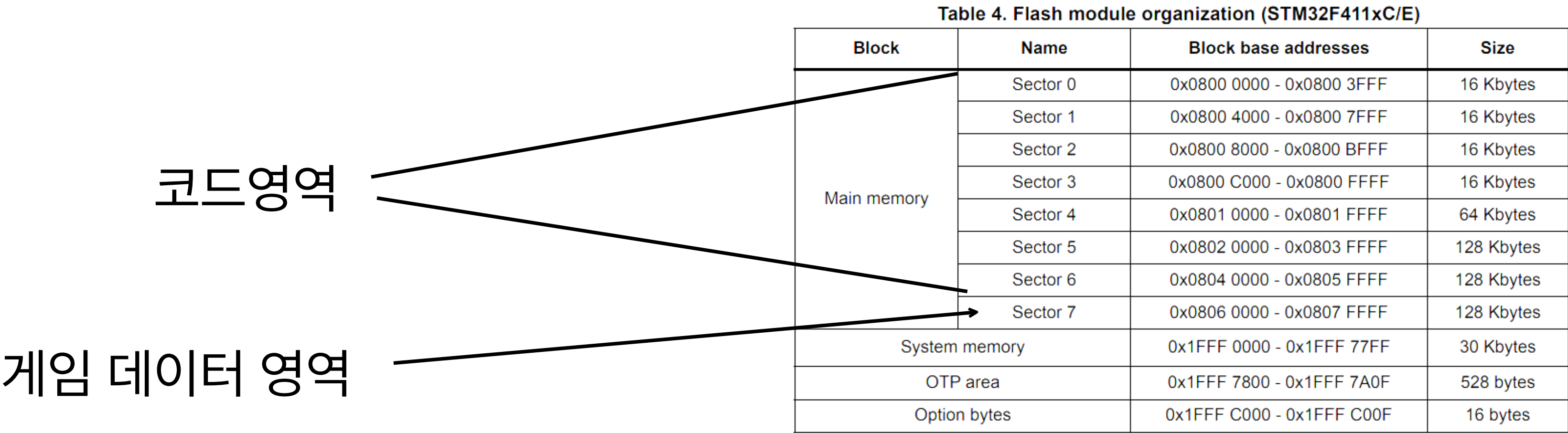
게임 데이터를 전원 종료 후에도 유지하기 위해
Flash의 Sector 7을 데이터 저장 영역으로 사용했습니다.

Flash는 데이터를 덮어쓰기 전에 Sector 단위 Erase가 필요하므로,
프로그램 영역과 분리하여 Sector 7을 전용 저장 공간으로 구성했습니다.

```
/* Entry Point */
ENTRY(Reset_Handler)

/* Specify the memory areas */
MEMORY
{
    RAM (xrw)      : ORIGIN = 0x20000000, LENGTH = 128K
    FLASH (rx)      : ORIGIN = 0x8000000, LENGTH = 384K
}
```

FLASH 384 KB → Sector 0~6만 코드 영역으로 제한



Flash header

게임 데이터 앞에 Header를 함께 저장하고,
Header 검증 후 Payload를 불러오는 구조로 설계했습니다.

Payload의 바이트들을 이용해 해시 값을 계산하고,
저장 당시 값과 읽은 뒤 다시 계산한 값을 비교해서
데이터 손상 여부를 확인했습니다.

```
#define USER_FLASH_SECTOR    FLASH_SECTOR_7
#define USER_FLASH_START_ADDR 0x08060000U
#define GDATA_MAGIC           0x47444154U // "GDAT" in ASCII
#define GDATA_VERSION         2U
#define GDATA_HASH_OFFSET_BASIS 2166136261U
#define GDATA_HASH_PRIME      16777619U
```

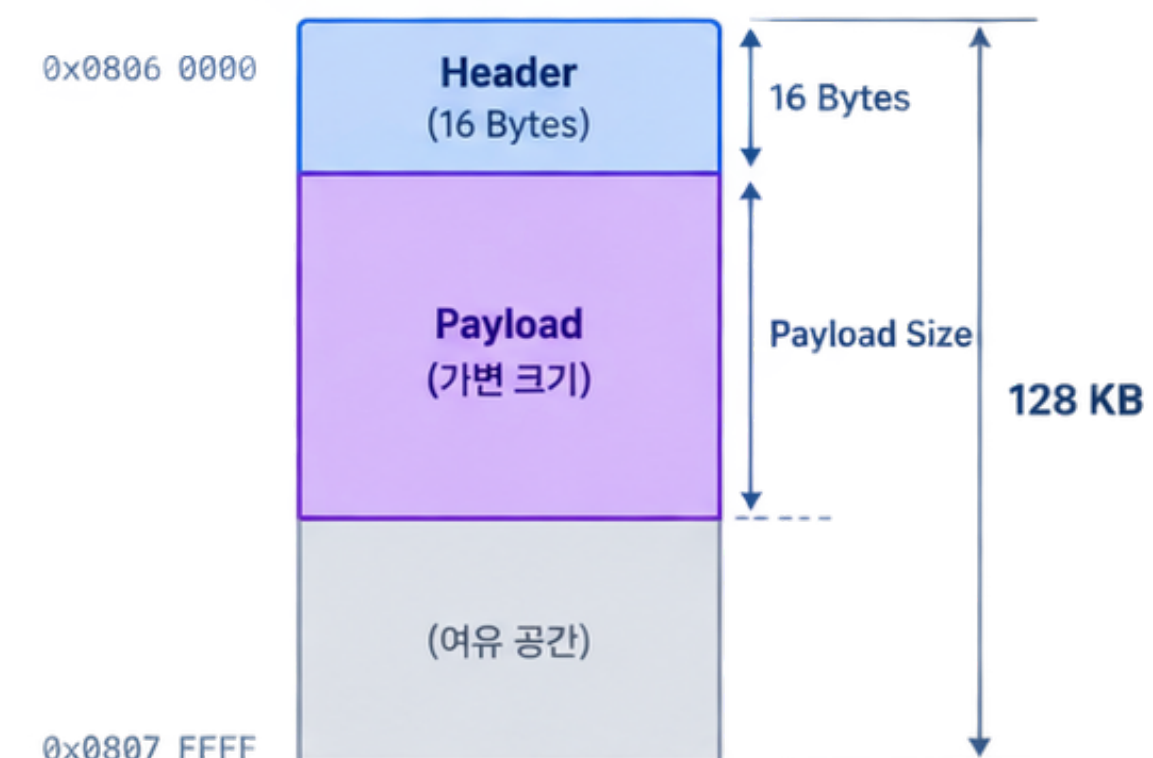
samana0104, 2 days ago | 1 author (samana0104)

```
typedef struct
{
    uint32_t magic;
    uint32_t version;
    uint32_t payloadSize;
    uint32_t checksum;
} GDataHeader;
```

Flash 저장 위치

Sector 7

(0x0806 0000 ~ 0x0807 FFFF)



데이터 저장 전 Sector 7을 먼저 Erase합니다.

데이터를 Word(32-bit) 단위로 순차 저장합니다.

```
bool SaveFlashData(void)
{
    GDataHeader *header = (GDataHeader *)dataWords;
    header->magic = GDATA_MAGIC;
    header->version = GDATA_VERSION;
    header->payloadSize = GDATA_PAYLOAD_SIZE;
    header->checksum = CalculateChecksum(data: Payload(), size: GDATA_PAYLOAD_SIZE);

    FLASH_EraseInitTypeDef erase = {0};
    uint32_t sectorError = 0U;
    erase.TypeErase = FLASH_TYPEERASE_SECTORS;
    erase.Sector = USER_FLASH_SECTOR;
    erase.NbSectors = 1U;
    erase.VoltageRange = FLASH_VOLTAGE_RANGE_3;

    if (HAL_FLASH_Unlock() != HAL_OK)
    {
        return false;
    }

    bool success = HAL_FLASHEx_Erase(&erase, &sectorError) == HAL_OK;
    for (size_t i = 0U; success && i < (GDATA_BUFFER_SIZE / sizeof(uint32_t)); ++i)
    {
        success = HAL_FLASH_Program(
            TypeProgram: FLASH_TYPEPROGRAM_WORD,
            Address: USER_FLASH_START_ADDR +
                (uint32_t)(i * sizeof(uint32_t)),
            Data: dataWords[i]) == HAL_OK;
    }

    if (HAL_FLASH_Lock() != HAL_OK)
    {
        success = false;
    }
}
```


FND(Flexible Numeric Display)

4자리 FND 동적 구동(Multiplexing)과 잔상 제어

세그먼트(A~G) 점등 패턴을 16진수 데이터로 사전 정의했습니다. Common Cathode 방식에 맞추어 출력 핀이 HIGH(1)일 때 발광 하도록 구성되었습니다.

입력된 인덱스에 해당하는 단일 자릿수의 공통 접지(Common) 핀만 LOW(RESET)로 활성화하여 한 번에 하나의 FND만 구동되도록 물리적 회로를 제어합니다.

```

19  /* Common Cathode 기준 폰트 테이블 */
20  static const uint8_t segmentTable[10] = {
21      0x3F, /* 0 (A B C D E F) */
22      0x06, /* 1 (B C) */
23      0x5B, /* 2 (A B D E G) */
24      0x4F, /* 3 (A B C D G) */
25      0x66, /* 4 (B C F G) */
26      0x6D, /* 5 (A C D F G) */
27      0x7D, /* 6 (A C D E F G) */
28      0x07, /* 7 (A B C) */
29      0x7F, /* 8 (A B C D E F G) */
30      0x6F /* 9 (A B C D F G) */
31  };
32
62  static void SelectDigit(uint8_t index)
63  {
64      HAL_GPIO_WritePin(DIGIT_PORT, DIG_SINGLE_PIN, (index == 0) ? GPIO_PIN_RESET : GPIO_PIN_SET);
65      HAL_GPIO_WritePin(DIGIT_PORT, DIG1_PIN, (index == 1) ? GPIO_PIN_RESET : GPIO_PIN_SET);
66      HAL_GPIO_WritePin(DIGIT_PORT, DIG2_PIN, (index == 2) ? GPIO_PIN_RESET : GPIO_PIN_SET);
67      HAL_GPIO_WritePin(DIGIT_PORT, DIG3_PIN, (index == 3) ? GPIO_PIN_RESET : GPIO_PIN_SET);
68      HAL_GPIO_WritePin(DIGIT_PORT, DIG4_PIN, (index == 4) ? GPIO_PIN_RESET : GPIO_PIN_SET);
69  }
70

```

FND(Flexible Numeric Display)

4자리 FND 동적 구동(Multiplexing)과 잔상 제어

새로운 자릿수를 켜기 직전, 모든 공통 접지를 차단하고 세그먼트 출력을 0V로 강제하여 고스트 현상을 차단합니다.

호출 종료 시마다 4개의 자릿수를 순차적으로 갱신합니다. 빠른 주기로 점멸을 반복하여 시각적으로 모든 자릿수가 동시에 켜져 있는 상태를 유지합니다.

```
115 void FndUpdate(void)
116 {
117     uint8_t digitVal;
118
119     if (!isInitialized)
120     {
121         return;
122     }
123
124     /* 잔상 방지를 위해 자릿수 전환 전 공통 접지 및 세그먼트 전압 완전 차단 */
125     SelectDigit(0xFF);
126     ShiftOutSegmentData(0x00); /* <--- 이 줄을 추가하여 0V 출력 강제 (잔상 제거) */
127
128     digitVal = displayDigits[currentDigitIndex];
129     ShiftOutSegmentData(segmentTable[digitVal]);
130
131     SelectDigit(currentDigitIndex);
132
133     currentDigitIndex = (currentDigitIndex + 1) % FND_TOTAL_DIGITS;
134 }
```

74HC595

74HC595 시프트 레지스터의 SIPO 동작

데이터 핀 설정 직후 클록(SHCP) 핀에 상승 에지(LOW ➡ HIGH)를 발생시켜 데이터를 내부 시프트 레지스터로 1비트씩 이동시킵니다.

비트 연산(>> i & 0x01)을 통해 원본 변수를 보존하면서, 8비트 데이터를 최상위 비트부터 1비트씩 추출해 데이터(DS) 핀에 인가합니다.

8비트 직렬 전송 완료 직후 래치(STCP) 핀에 펄스를 발생시켜, 내부 레지스터에 적재된 데이터를 외부 핀(Q0~Q7)으로 일괄 갱신합니다.

```

37  static void ShiftOutSegmentData(uint8_t segByte)
38  {
39      /* Q7(DP)부터 Q0(A)까지 MSB First 전송 */
40      for (int i = 7; i >= 0; i--)
41      {
42          HAL_GPIO_WritePin(FND_CLOCK_PORT, FND_CLOCK_PIN, GPIO_PIN_RESET);
43
44          if ((segByte >> i) & 0x01)
45          {
46              HAL_GPIO_WritePin(FND_DATA_PORT, FND_DATA_PIN, GPIO_PIN_SET);
47          }
48          else
49          {
50              HAL_GPIO_WritePin(FND_DATA_PORT, FND_DATA_PIN, GPIO_PIN_RESET);
51          }
52
53          HAL_GPIO_WritePin(FND_CLOCK_PORT, FND_CLOCK_PIN, GPIO_PIN_SET);
54      }
55
56      /* 래치 클록 발생 (출력 갱신) */
57      HAL_GPIO_WritePin(FND_LATCH_PORT, FND_LATCH_PIN, GPIO_PIN_RESET);
58      HAL_GPIO_WritePin(FND_LATCH_PORT, FND_LATCH_PIN, GPIO_PIN_SET);
59      HAL_GPIO_WritePin(FND_LATCH_PORT, FND_LATCH_PIN, GPIO_PIN_RESET);
60  }

```




트러블 슈팅

CubeMX 초기 환경 설정 시 버전 문제

```
16 Mcu.Pin0=PC13-ANTI_TAMP
17 Mcu.Pin1=PC14-OSC32_IN
18 Mcu.Pin10=PB3
19 Mcu.Pin11=VP_SYS_VS_Systick
20 Mcu.Pin2=PC15-OSC32_OUT
21 Mcu.Pin3=PH0 - OSC_IN
22 Mcu.Pin4=PH1 - OSC_OUT
23 Mcu.Pin5=PA2
24 Mcu.Pin6=PA3
25 Mcu.Pin7=PA5
26 Mcu.Pin8=PA13
27 Mcu.Pin9=PA14
28 Mcu.PinsNb=12
29 Mcu.ThirdPartyNb=0
30 Mcu.UserConstants=
31 Mcu.UserName=STM32F411RETx
32 MxCube.Version=6.18.1
33 MxDb.Version=DB.6.0.181
34 NVIC.BusFault_IRQn=true\0\0:false:false:true:true:false:false
35 NVIC.DebugMonitor_IRQn=true\0\0:false:false:true:true:false:fal
36 NVIC.ForceEnabledDMAVector=true
37 NVIC.HardFault_IRQn=true\0\0:false:false:true:true:false:false
38 NVIC.MemoryManagement_IRQn=true\0\0:false:false:true:true:false\
39 NVIC.NonMaskableInt_IRQn=true\0\0:false:false:true:true:false\f
40 NVIC.PendSV_IRQn=true\0\0:false:false:true:false:false:false
41 NVIC.PriorityGroup=NVIC_PRIORITYGROUP_0
42 NVIC.SVCall_IRQn=true\0\0:false:false:true:false:false:false
43 NVIC.SysTick_IRQn=true\0\0:true:false:true:true:true:false
44 NVIC.UsageFault_IRQn=true\0\0:false:false:true:true:false:false
45 PA13.GPIOParameters=GPIO_Label
```

```
16 Mcu.Pin0=PC13-ANTI_TAMP
17 Mcu.Pin1=PC14-OSC32_IN
18 Mcu.Pin10=PB3
19 Mcu.Pin11=VP_SYS_VS_Systick
20 Mcu.Pin2=PC15-OSC32_OUT
21 Mcu.Pin3=PH0 - OSC_IN
22 Mcu.Pin4=PH1 - OSC_OUT
23 Mcu.Pin5=PA2
24 Mcu.Pin6=PA3
25 Mcu.Pin7=PA5
26 Mcu.Pin8=PA13
27 Mcu.Pin9=PA14
28 Mcu.PinsNb=12
29 Mcu.ThirdPartyNb=0
30 Mcu.UserConstants=
31 Mcu.UserName=STM32F411RETx
32 MxCube.Version=6.18.0
33 MxDb.Version=DB.6.0.181
34 NVIC.BusFault_IRQn=true\0\0:false:false:true:true:false:false
35 NVIC.DebugMonitor_IRQn=true\0\0:false:false:true:true:false:fal
36 NVIC.ForceEnabledDMAVector=true
37 NVIC.HardFault_IRQn=true\0\0:false:false:true:true:false:false
38 NVIC.MemoryManagement_IRQn=true\0\0:false:false:true:true:false\
39 NVIC.NonMaskableInt_IRQn=true\0\0:false:false:true:true:false\f
40 NVIC.PendSV_IRQn=true\0\0:false:false:true:false:false:false
41 NVIC.PriorityGroup=NVIC_PRIORITYGROUP_0
42 NVIC.SVCall_IRQn=true\0\0:false:false:true:false:false:false
43 NVIC.SysTick_IRQn=true\0\0:true:false:true:true:true:false
44 NVIC.UsageFault_IRQn=true\0\0:false:false:true:true:false:false
45 PA13.GPIOParameters=GPIO_Label
```

문제

CubeMX의 .ioc 파일이 열리지 않는 문제

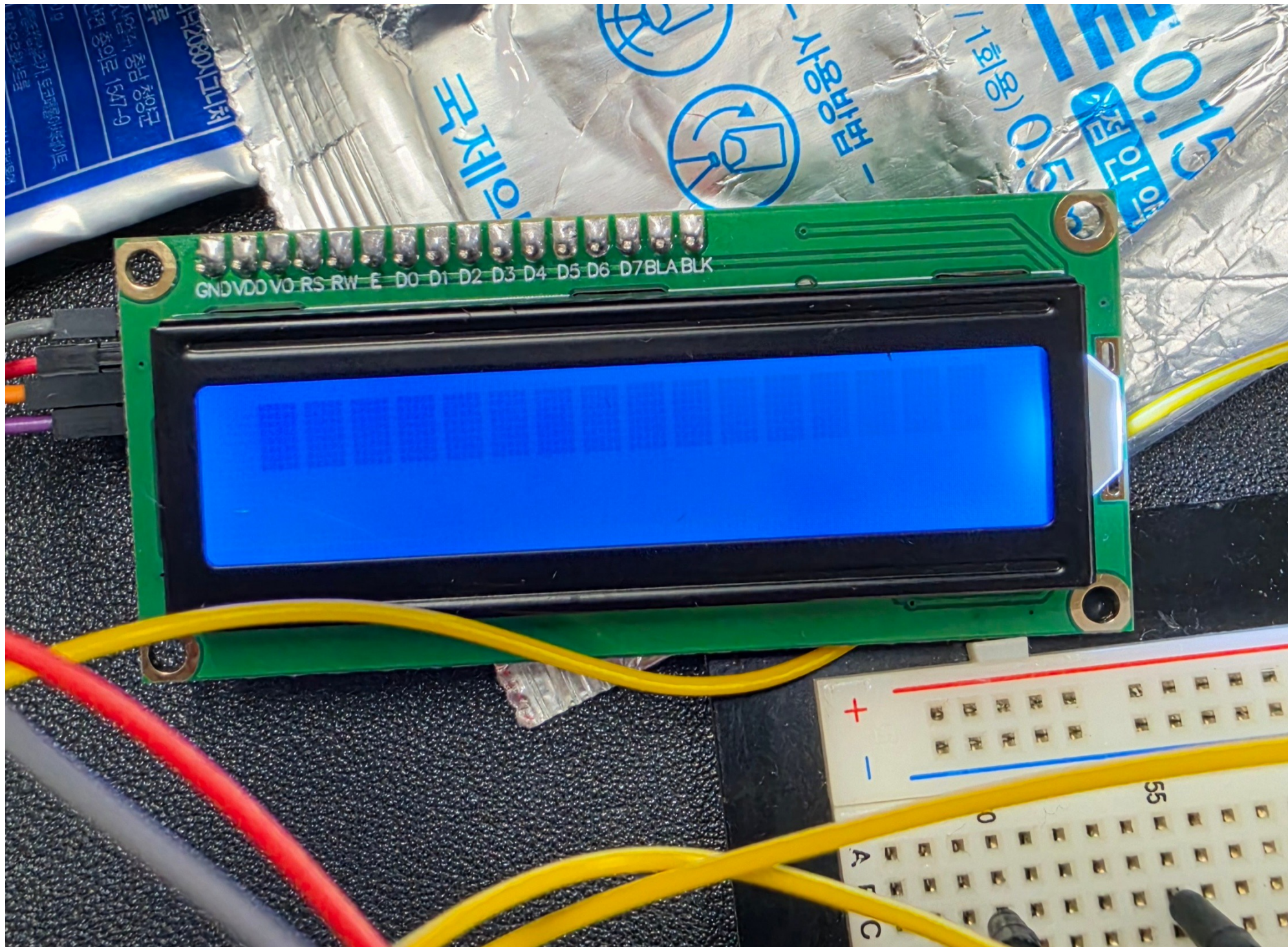
원인

개인 PC에서 개발 환경을 구성한 후 다른 PC에서 프로젝트를 실행하는 과정에서 CubeMX 버전 차이로 인한 호환성 문제가 발생했습니다.

해결

.ioc 파일은 Key-Value 형태의 텍스트 기반 설정 파일이므로 CubeMX 버전 정보를 직접 수정하여 문제를 해결했습니다.

LCD1602 전원 불안정 시 출력 복구 문제



문제

전원 불안정 발생 시 LCD 출력이 사라지고 정상적으로 복구되지 않는 현상

원인

전원 변동으로 인해 LCD 컨트롤러가 비정상 상태에 진입한 것으로 추정했습니다.

해결

LCD 데이터를 주기적으로 재전송하여 출력을 복구하고, HAL_Delay() 대신 Tick 기반 논블로킹 처리로 변경하여 메인 루프의 지연을 최소화했습니다.

FND가 심하게 깜빡이던 문제

문제

숫자는 정상적으로 출력되었지만, FND가 심하게 깜빡여 보기 불편했습니다.

원인

기존에는 FND 갱신을 Main Loop에서 처리했습니다. 그런데 게임 로직이나 LCD 출력 처럼 처리 시간이 긴 작업이 발생하면 Main Loop의 주기가 불규칙해졌고, 그 영향으로 FND 갱신 간격도 일정하지 않아 깜빡임이 발생했습니다.

해결

FND 갱신을 SysTick 인터럽트로 분리하여 1ms 주기로 처리했습니다.
게임 로직의 처리 시간과 관계없이 일정한 주기로 갱신되면서
안정적인 FND 출력을 구현했습니다.

한줄 소감



변한빛(팀장)

소프트웨어적으로 해결했던 부분을
하드웨어로 해결해서 최적화를 더 끌어낼 수 있지 않았을까 아쉬움이 남습니다.



최선호

보드와 모듈 간의 연결을 좀 더 깔끔하게 정리하여 게임플레이에 있어 거슬리지 않게
할 수 있지 않았을까 싶습니다.



김민식

복잡한 배선을 74HC165 같은 칩을 이용해 좀 더 최적화 할 수 있지 않았을까
하는 아쉬움이 있는 것 같습니다.



남기범

- LCD에 두더지를 직접 표현하지 못한 점이 아쉬움
- 구현 한계로 LED를 활용해 두더지 위치를 표현
- 추후 OLED를 활용해 게임성을 높이고 싶음

맡은 부분



변한빛(팀장)

- 게임 기획 및 설계
- 프로젝트 환경 및 빌드 시스템 구축
- 일정 관리 및 작업 배분
- 초기 게임 로직 개발
- 디버깅 CLI 콘솔 개발
- 부저 기반 사운드 모듈 개발
- Flash 기반 저장·로드 기능 개발



최선호

- LED 동작 구현
- FND 모듈 동작 구현
- 외부 전원 구동
- 최종 보드 배선 및 제작



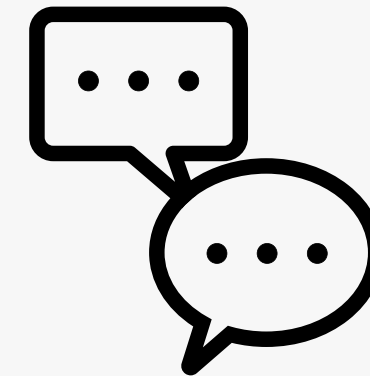
김민식

- 스위치, LED 저항 최적화
- 74HC595 전원 전압 안정화
- 게임 버그 발견 및 최적화 제안
- 사진, 영상, 발표
- tinkercad를 이용한 회로도 작성



남기범

- 조이스틱 입력 동작 구현
- LCD1602 출력 기능 구현
- Stage 별 게임 진행 로직 수정
- 기능 모듈 간 코드 연동 및 통합
- 게임 동작 검증 및 디버깅



질의응답 시간입니다.

누가 최고 점수를 기록할까?

실제 보드로 플레이하여 팀별로 최고 점수를 찍은 팀에게 상품을 드립니다.

CHALLENGE RULE

1. 각 팀 조장 게임 1회 플레이
2. 점수 기록
3. 1등 팀에게 경품 제공



SCORE

1. 9999



Thank you

Team. 똑배기 브레이커